Software Design Document

Memphis Music Initiative - Data Management System

1. Introduction....2
1.1. Purpose....2
1.2. Objectives....2
1.2.1. Efficient Information Management....2
1.2.2. Central Platform....2
1.2.3. Enrollment Tracking....2
1.2.4. Progress Monitoring....2
1.2.5. Survey Collection....2
1.2.6. Data Analysis....3
1.3. Scope....3
1.4. Audience....3
1.5. Alignment with MMI's Goals....3
1.6. Design Milestones....3
2. Technical Architecture....3
2.1. System Architecture....5
2.2. Technology Stack....6
2.2.1. Docker....6
2.2.2. Docker Swarm....6
2.2.3. Linode Virtual Private Server....7
2.2.4. HTTPS....7
2.2.5. Traefik....8
2.2.6. Django (Python)....8
2.2.7. Adminer....9
2.2.8. NGinx....9
2.2.9. PostgreSQL....10
2.3. Database Design....10
2.3.1. Organizational Hierarchy....11
2.3.2. Form Design and Assignment....13
2.3.3. User Organization....14
2.3.4. Data Collection with Form Entries....15
2.3.5. Reports and Data Dashboards....16
2.4. User Interface....18
2.4.1. Login Screen....18
2.4.2. MMI User Home Screen....19
2.4.3. Partner User Home Screen....20

2.4.4. Parent and Client User Home Screens....21
2.4.5. Manage Partners Screen....22
2.4.6. Add/Edit Partner Screen....23
2.4.7. Add/Edit User Screen....24
2.4.8. Manage Programs Screen....25
2.4.9. Add/Edit Program Screen....26
2.4.10. Add/Edit Client Screen....27
2.4.11. Manage Activities Screen....28
2.4.12. Add/Edit Activity Screen....29
2.4.13. Manage Sessions Screen....30
2.4.14. Add/Edit Session Screen....31
2.4.15. Manage Forms Screen....32
2.4.16. Add/Edit Form Screen....33
2.4.17. Form Assignment Screen....34
2.4.18. View Form Entry....35
2.4.19. Complete Assigned Form Screen....36
2.4.20. Reports Menu....37
2.4.21. View Report Screen....38
2.4.22. MMI/Partner/Program Dashboard Screen....39
2.5. Sample Workflows....40
2.5.1. Setting up a New Partner and Program....40
2.5.2. Adding Activities and Sessions to a Program....41
2.5.3. Conducting a Retrospective Survey....41
2.5.4. Completing a Form (Client/Parent)....41
2.6. Security Measures....42
2.7. Scalability and Performance....43

DRAFT

1. Introduction

1.1. Purpose

The purpose of this Software Design Document (SDD) is to outline the objectives, scope, and architecture of the proposed software solution, referred to as the Data Management System (DMS). The DMS project aims to address critical needs within the Memphis Music Initiative (MMI) and its partner programs, with the overarching goal of enhancing program management, streamlining enrollment tracking, and improving progress monitoring.

1.2. Objectives

1.2.1. Efficient Information Management

The primary objective of the DMS is to establish an efficient system for collecting, storing, and managing information pertaining to students enrolled in MMI partner programs. This includes demographic data, enrollment history, program participation records, and academic progress.

1.2.2. Central Platform

The DMS will serve as a centralized platform for MMI and its partner organizations, allowing them to access, update, and utilize student information seamlessly. This centralization will enhance collaboration, reduce redundancy, and improve data accuracy.

1.2.3. Enrollment Tracking

The DMS will enable real-time tracking of student enrollment, making it easier to monitor program participation and identify trends in enrollment numbers over time. This feature will aid in resource allocation and program planning.

1.2.4. Progress Monitoring

The system will provide tools for tracking and assessing the academic progress and personal development of students within MMI partner programs. This information will be valuable for making data-driven decisions to improve program effectiveness.

1.2.5. Survey Collection

One of the key functionalities of the DMS is to facilitate the collection of surveys from program participants. These surveys will be designed to gather valuable feedback and

DRAFT

insights from students, instructors, and program administrators. The data collected will be used for grant reporting and to enhance program quality.

1.2.6. Data Analysis

The DMS will incorporate data analysis tools to process the collected information, enabling MMI and partner organizations to derive actionable insights. These insights will inform program improvements and support evidence-based decision-making.

1.3. Scope

The scope of this project encompasses the design, development, and implementation of the DMS software. It includes defining user roles and access privileges, designing a user-friendly interface, and implementing robust data security measures to safeguard sensitive student information. Additionally, the project will entail the creation of custom survey templates and data analysis modules tailored to MMI's specific needs.

1.4. Audience

This SDD is intended for developers, project managers, stakeholders, and any individuals involved in the planning, development, and deployment of the DMS. It serves as a comprehensive guide to understanding the system's architecture, functionality, and technical requirements.

1.5. Alignment with MMI's Goals

The DMS project aligns closely with MMI's mission to empower young musicians and support music education. By enhancing data collection, analysis, and reporting capabilities, the DMS will aid in optimizing program delivery, ensuring accountability, and ultimately improving the outcomes and experiences of students participating in MMI partner programs.

The DMS will provide valuable insights and metrics that can be used to make informed decisions and drive continuous improvement in Grant-Making and Community Building (GM&CB) program design and implementation. Additionally, DMS will streamline processes, facilitate effective communication, and enable to track progress and measure the impact of GM&CB programs more accurately.

1.6. Design Milestones

Through this Software Design Document, the aim is to provide a clear roadmap for the development of the DMS, ensuring that it fulfills its purpose effectively and contributes to the success of MMI and its partner organizations. This section describes the planned

DRAFT

major milestones in the system design, supported by the requirements as described in the System Requirements Specification.

Milestone Number	Milestone Description	Anticipated Date
1	Data Model Complete	11/02/2023
2	Home Screens Complete	11/17/2023
3	Manage Screens Complete	11/30/2023
4	Add/Edit Screens Complete	12/14/2023
5	Form Builder Complete	12/29/2023
6	Form Entry Collection Complete	12/29/2023
7	Reports Complete	01/12/2023
8	Dashboards Complete	01/12/2023

2. Technical Architecture

The Technical Architecture section of this Software Design Document (SDD) delves into the foundational elements and structures that underpin the Data Management System (DMS) project. It provides a comprehensive overview of the software's technical design, outlining the key components, technologies, and interactions that will enable the DMS to fulfill its objectives effectively.

The technical architecture serves as the backbone of the DMS, shaping its scalability, performance, security, and flexibility. It lays the groundwork for robust data management, seamless user experiences, and the ability to adapt to evolving requirements. In this section, we will explore the architecture from both a high-level perspective and a detailed technical viewpoint, addressing the following key aspects:

1. **System Architecture**: This section will define the system's overall structure, including its various modules, components, and their interrelationships. This section will also elucidate how data flows through the system, highlighting key data sources, processing stages, and storage mechanisms.

2. **Technology Stack**: This section will identify the technologies and frameworks chosen for the DMS development. This includes programming languages, databases, web frameworks, and other critical tools that contribute to the system's functionality and performance.

DRAFT

3. **Database Design**: The architecture section will delve into the design of the database schema, emphasizing the organization of data, relationships between entities, and data integrity measures.

4. **User Interfaces**: This section will discuss the user interface (UI) design, outlining how the system's interface elements facilitate user interaction. This includes the layout, navigation, and accessibility features that contribute to a user-friendly experience.

5. **Sample Workflows**: This section will discuss a small number of user workflows in the system, showing how a user may navigate from screen to screen in order to accomplish common tasks.

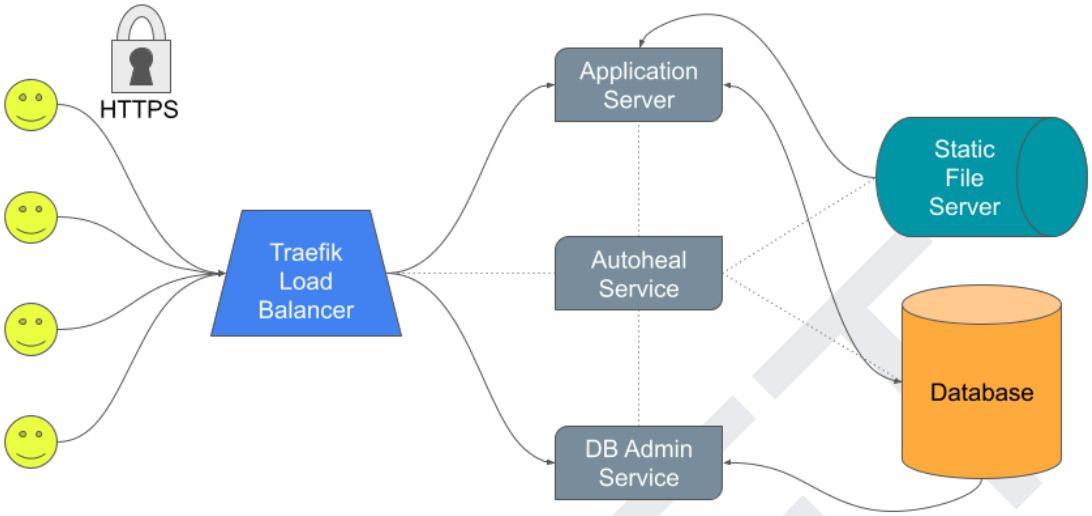
6. **Security Measures**: Security is paramount in handling sensitive student information. This section will detail the security measures and protocols implemented to protect data at rest and in transit, as well as user authentication and authorization mechanisms.

7. **Scalability and Performance**: The architecture section will address strategies for scaling the DMS to accommodate increasing data loads and user traffic. It will also outline performance optimization techniques to ensure rapid response times.

By comprehensively exploring these technical aspects, this document aims to offer a clear and detailed blueprint for the development team and stakeholders. It serves as a vital reference point throughout the development lifecycle, ensuring that the DMS is built in a cohesive, efficient, and maintainable manner. Additionally, it aligns the technical architecture with the overarching purpose of the project: to enhance data management, program effectiveness, and the overall impact of the Memphis Music Initiative and its partner organizations.

DRAFT

2.1. System Architecture



This diagram provides a high-level overview of the overall system which operates as described in this section.

- All users access the publicly available services (DMS Application, DB Admin Service) via an encrypted HTTPS connection. Any attempt to access the system via an un-encrypted HTTP connection will be re-routed to HTTPS.
- The system consists of a number of microservices deployed in unison. Each service is provided by one or more 'containers', orchestrated via the Traefik application in a single container, which serves as both an entry point and load balancer. All containers will be hosted on a Linode Virtual Private Server.
- The DMS Application consists of one or more containers which provide the main user interface and business logic layer between the user and the underlying database.
- The Autoheal Service consists of a single container which monitors the other microservices and attempts to re-start any service that fails for any reason.
- The DB Admin Service consists of one or more containers providing an administrative user interface directly into the underlying database.
- The Static File Server consists of one or more containers that each houses an NGinx web server that supplies any and all static assets (such as images) accessible via the DMS Application.
- The Database consists of a single container that houses all the application data accessible via the DMS Application or DB Admin Service.

DRAFT

2.2. Technology Stack

2.2.1. Docker

Docker is an open-source platform for developing, shipping, and running applications. It enables developers to package their applications with all of the dependencies they need, into a standalone container that can run on any operating system. This makes it easy to deploy and manage applications, and to scale them up or down as needed.

Docker is used by millions of developers and organizations around the world, including some of the largest and most innovative companies, such as Google, Amazon, and Netflix. It is also used by many small and medium-sized businesses, as well as startups.

Docker offers a number of benefits, including:

- Portability: Docker containers can run on any operating system, making them highly portable. This makes it easy to deploy applications to different environments, such as development, testing, and production.
- Scalability: Docker containers can be easily scaled up or down, making it easy to handle changes in demand. This is ideal for applications that experience spikes in traffic or that need to be scaled up for seasonal events.
- Efficiency: Docker containers are very efficient, as they share the underlying operating system kernel. This means that you can run more containers on a single server, which can save you money on hardware costs.
- Security: Docker containers are isolated from each other, which helps to improve security. This is important for running applications that handle sensitive data.

2.2.2. Docker Swarm

Docker Swarm is a container orchestration platform that helps to automate the deployment and management of containerized applications. It is a native Docker product, and it is built on top of the Docker Engine.

Docker Swarm works by grouping together multiple Docker hosts into a single cluster. This cluster is then managed by a set of manager nodes, which are responsible for scheduling tasks to worker nodes and for ensuring that the cluster remains healthy.

Docker Swarm offers a number of benefits for organizations that are running containerized applications, including:

- Scalability: Docker Swarm can be scaled up or down to meet the needs of your application, making it ideal for running both small and large workloads.

DRAFT

- High availability: Docker Swarm is designed to be highly available, and it can continue to operate even if some of the nodes in the cluster fail.
- Security: Docker Swarm provides a number of security features, such as role-based access control and network isolation.
- Ease of use: Docker Swarm is easy to use, and it can be managed using the Docker CLI or the Docker Swarm GUI.

2.2.3. Linode Virtual Private Server

Linode Virtual Private Servers (VPS) are a type of cloud computing service that provides dedicated computing resources to users. Linode VPS are created on top of bare-metal servers, which means that users have exclusive access to the resources allocated to their VPS.

Linode VPS offer a number of benefits, including:

- Scalability: Linode VPS can be easily scaled up or down to meet the needs of your application. This makes them ideal for applications that experience spikes in traffic or that need to be scaled up for seasonal events.
- Performance: Linode VPS offer high performance and low latency. This is because they are created on top of bare-metal servers and because they use a variety of performance optimization techniques.
- Security: Linode VPS are secure and isolated from each other. This helps to protect your data from being accessed by other users.
- Control: Linode VPS gives you full control over your computing environment. You can choose your own operating system, install your own software, and configure your VPS to meet your specific needs.

2.2.4. HTTPS

HTTPS, or Hypertext Transfer Protocol Secure, is a secure version of the HTTP protocol that is used for transferring data over the web. It uses encryption to protect data from being intercepted or modified during transmission.

HTTPS is used by all major websites, including Google, Amazon, and Facebook. It is also used by many businesses and organizations to protect their customer data.

HTTPS offers a number of benefits, including:

- Security: HTTPS encrypts data in transit, which helps to protect it from being intercepted or modified by attackers.
- Trust: HTTPS websites display a lock icon in the browser address bar, which indicates to users that the website is secure and that their data is protected.

DRAFT

- SEO: Google gives preference to HTTPS websites in its search results. This means that HTTPS websites are more likely to rank higher in search results than websites that do not use HTTPS.

2.2.5. Traefik

Traefik is a modern reverse proxy and load balancer that makes routing requests to microservices simple, yet powerful. It automatically detects your services and provides dynamic routing based on service tags and labels, eliminating the need for manual configuration.

Traefik offers a number of benefits, including:

- Simplicity: Traefik is easy to use and configure. It automatically detects your services and provides dynamic routing based on service tags and labels.
- Flexibility: Traefik can be used in a variety of environments, including on-premises, cloud, and Kubernetes.
- Performance: Traefik is lightweight and fast. It can handle a large volume of traffic without any performance problems.
- Security: Traefik provides a number of security features, such as authentication, authorization, and HTTPS support.

2.2.6. Django (Python)

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It is built on top of the Python standard library and provides a number of features, including:

- A full-featured object-relational mapper (ORM)
- A powerful templating system
- A built-in user authentication system
- A robust admin interface
- A number of other features that make web development easier and more efficient

Django offers a number of benefits, including:

- Rapid development: Django is designed to help developers build web applications quickly and efficiently. It provides a number of features that make web development easier, such as an ORM, a templating system, and a built-in user authentication system.
- Scalability: Django is scalable and can handle large amounts of traffic. It is used by some of the busiest websites in the world, such as Instagram and Pinterest.
- Security: Django is secure and provides a number of features to help protect web applications from attack.
- Community support: Django has a large and active community that provides support and resources for developers.

DRAFT

2.2.7. Adminer

Adminer is a full-featured database management tool written in PHP. It can be used to manage a variety of database systems, including PostgreSQL.

Adminer is a web-based tool, so it can be accessed from anywhere with a web browser. It is also a very lightweight tool, so it can be run on even the most modest of hardware.

Adminer provides a wide range of features for managing PostgreSQL databases, including:

- Creating and dropping databases
- Creating and dropping tables
- Creating and dropping views
- Inserting, updating, and deleting data
- Running SQL queries
- Managing users and roles
- Exporting and importing data

2.2.8. NGinx

NGINX is an open-source web server and reverse proxy that is known for its high performance, scalability, and reliability. It is used by some of the largest and most popular websites in the world, such as Netflix, Airbnb, and Hulu.

NGINX can be used for a variety of purposes, including:

- Web serving: NGINX can be used to serve static and dynamic web content.
- Reverse proxying: NGINX can be used to reverse proxy traffic to other web servers, such as Apache or Tomcat.
- Load balancing: NGINX can be used to load balance traffic across multiple web servers.
- Caching: NGINX can be used to cache static and dynamic web content, which can improve the performance of websites.

NGINX offers a number of benefits, including:

- High performance: NGINX is designed to be very fast and can handle a large volume of traffic.
- Scalability: NGINX can be scaled horizontally to handle even the most demanding workloads.
- Reliability: NGINX is very reliable and can withstand even the most severe traffic spikes.
- Security: NGINX provides a number of security features, such as authentication, authorization, and DDoS protection.

DRAFT

2.2.9. PostgreSQL

PostgreSQL is a powerful, open-source object-relational database system (ORDBMS) that has been in development for over 35 years. It is known for its reliability, scalability, and feature richness. PostgreSQL is used by a wide range of organizations, including Fortune 500 companies, small businesses, and government agencies.

PostgreSQL offers a number of benefits, including:

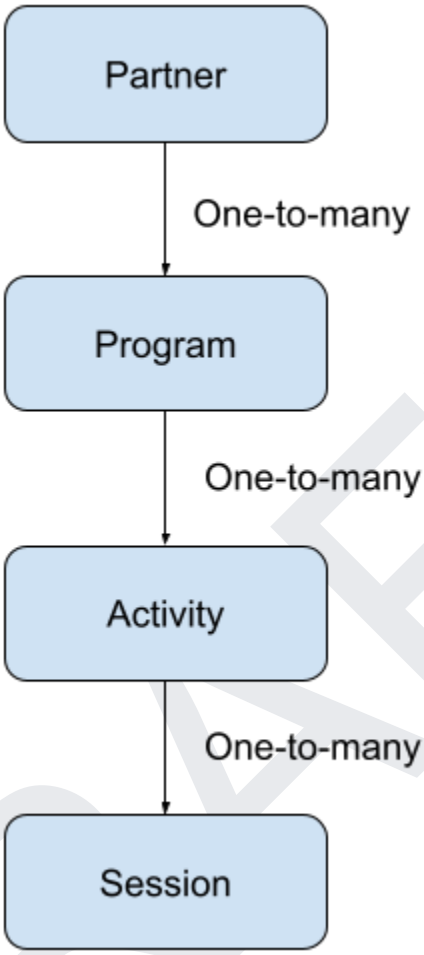
- Reliability: PostgreSQL is a highly reliable database system that can withstand even the most demanding workloads. It has a proven track record of stability and performance.
- Scalability: PostgreSQL can be scaled to meet the needs of even the largest organizations. It can be used to manage databases of all sizes, from small development databases to massive production databases.
- Feature richness: PostgreSQL offers a wide range of features, including support for ACID transactions, complex queries, and stored procedures. It also supports a variety of data types, including relational data, JSON, and XML.
- Open source: PostgreSQL is an open-source database system, which means that it is free to use and modify. This makes it a cost-effective solution for organizations of all sizes.

2.3. Database Design

This section will describe the various relationships between logical entities in the data store. Each logical entity serves as a convenient abstraction for coherent groupings of data collected and stored by the system, though the data for each entity may not necessarily be stored in a single database table. This section does not serve as a full Entity Relationship Diagram nor Data Dictionary for the DMS, but serves to demonstrate the strategy for organizing data within the system.

DRAFT

2.3.1. Organizational Hierarchy



DRAFT

The DMS organizational hierarchy is straightforward and consists of the following entities and relationships:

- Partner: The parent entity for all data in the system. It may represent an actual partner agency or simply an organizational tool for grouping one or more programs.
- Program: A conceptual representation of a program of services or activities offered by a partner. It provides an organizational tool for grouping one or more activities.
- Activity: A conceptual representation of a particular class, offering, or activity offered by a program. It may be conducted across one or more sessions.
- Session: A single part of a multi-part activity. For example, a leadership class that is offered over the course of several weeks. It is the lowest level of the organizational hierarchy and does not have any dependents.

Relationships Between Entities

- A partner may have one or more programs.
- A program may have one or more activities.
- An activity may have one or more sessions.

Example

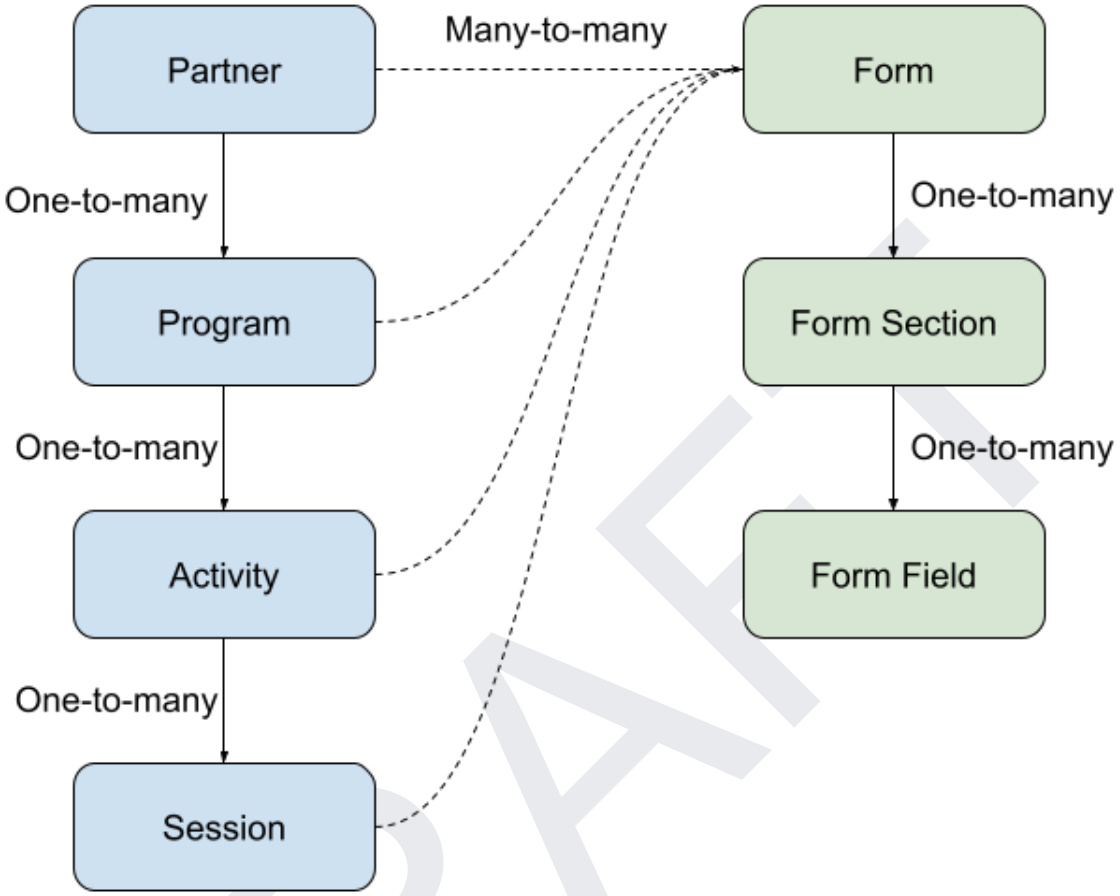
Here is an example of the DMS organizational hierarchy for an orchestra that provides music education:

- Partner: The orchestra itself.
- Program: A specific music education program offered by the orchestra, such as a youth orchestra program, a summer music camp, or a private lesson program.
- Activity: A particular class or offering within a music education program, such as a violin class, a choir rehearsal, or a music theory lesson.
- Session: A single part of a multi-part activity, such as a weekly violin lesson or a monthly choir rehearsal.

The orchestra offers a youth orchestra program for students ages 6-18. The youth orchestra program has multiple activities, such as a string orchestra, a wind ensemble, and a percussion ensemble. Each activity may have multiple sessions. For example, the string orchestra meets once a week for two hours.

DRAFT

2.3.2. Form Design and Assignment



Data collection in the DMS is managed through the use of forms. Forms are logical entities that organize data collected from clients and programs. Forms can be associated with any level of the organizational hierarchy, including partners, programs, activities, and sessions. The same form can also be shared among different partners, programs, activities, or sessions.

Relationships Between Entities

- One or more forms can be associated with one or more partners.
- One or more forms can be associated with one or more programs.
- One or more forms can be associated with one or more activities.
- One or more forms can be associated with one or more sessions.
- One or more form sections can be associated with a single form.
- One or more form fields can be associated with a single form section.

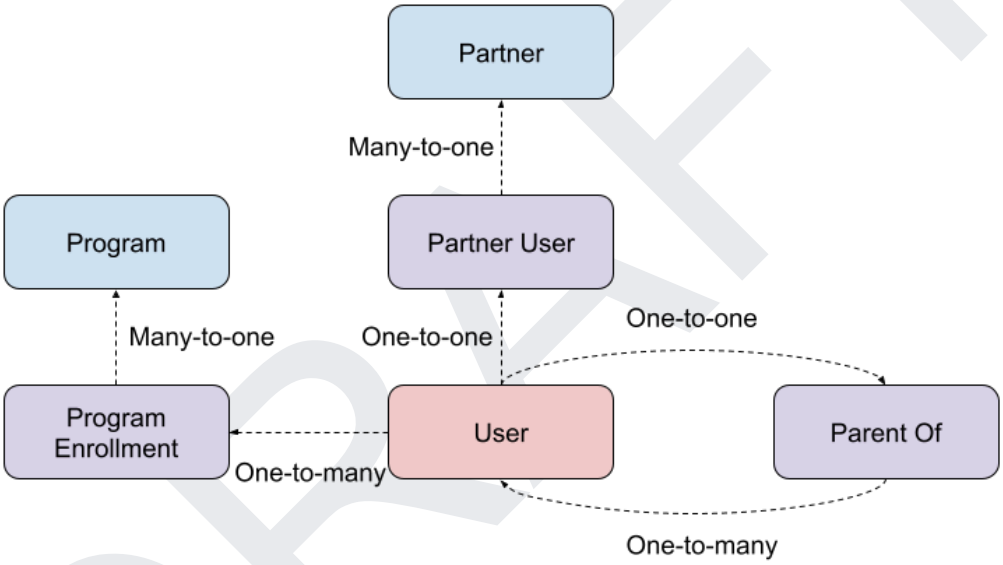
Example

A partner may want to collect a client satisfaction form once per year for every client enrolled in a specific program. The form may also be used to collect feedback from clients who participate in a particular session of an activity. In this case, the client satisfaction form would be associated with both the program and the session. The form may consist of one or more sections, each with one or more fields.

Note

While a form may be shared between partners, the individual form entries are not. This means that each partner will have its own copy of the form data, and changes made by one partner will not be reflected in the form data of other partners.

2.3.3. User Organization



There are four user roles in the DMS:

- MMI Users: MMI users are not associated with any partner and have full access to all partner data.
- Partner Users: Partner users are associated with a single partner and can only view and modify data associated with that partner.
- Client Users: Client users are associated with one or more programs via program enrollment and can view or complete any forms assigned to them.
- Parent Users: Parent users are associated with one or more client users and can view or modify any forms associated with those client users.

Relationships Between Entities

- A user may not be associated with any partner, program, or other user (MMI user).

DRAFT

- A user may be associated with a partner (partner user).
- A user may be associated with one or more programs (client user).
- A user may be associated with one or more other users (parent user).

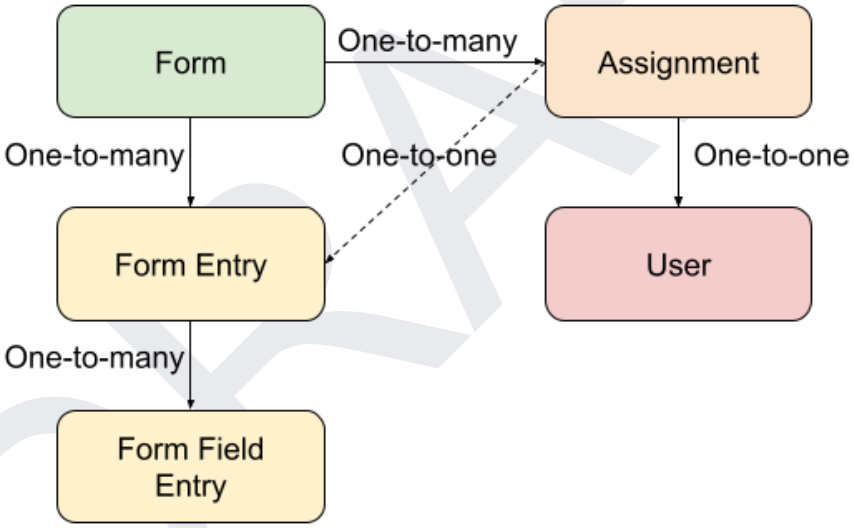
Example

An MMI user can view and modify all partner data, such as program information, activity information, and session information. A partner user can only view and modify data associated with their assigned partner.

A client user can view or complete any forms assigned to them. For example, a client user may be assigned a form to enroll in a program or to provide feedback on an activity.

A parent user can view or modify any forms associated with their assigned client users. For example, a parent user may be assigned a form to provide consent for their child to participate in a program or to update their child's contact information.

2.3.4. Data Collection with Form Entries



Forms are templates for collecting data about partners, programs, activities, sessions, and/or users. Form entries are the entities that hold the data from completed forms.

Relationships Between Entities

- Each form entry is associated with corresponding form field entries, which hold the data from the form fields that were active on the filled form at the time the data was collected.
- Form field entries reflect the fields on the form at the time of entry and will not be updated with new fields or have entries for deleted fields removed.
- Each form may have one or more associated form entries, and each form entry may have one or more form field entries.

DRAFT

- Each form and form entry may also be associated with a user through an assignment, indicating that the user needs to fill out or has filled out the form.

Example

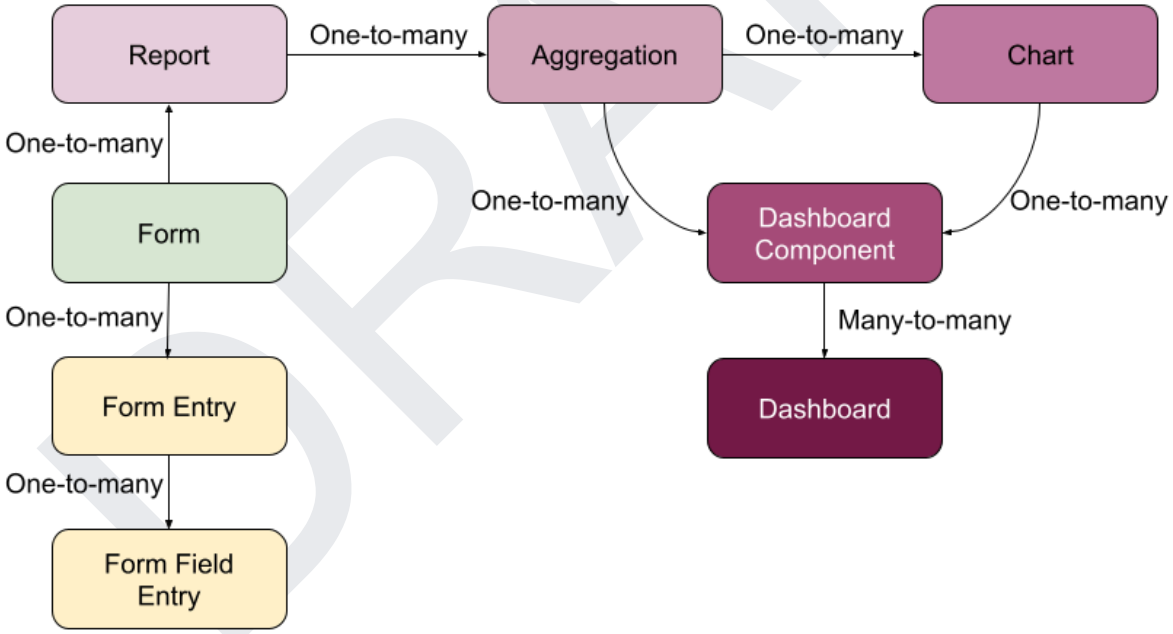
A partner may create a form to collect client information for a new program. The form may have fields for the client's name, address, phone number, and email address.

When a client fills out the form, a form entry is created to store the client's data. The form entry is associated with the form that was used to collect the data, and each form field entry is associated with the corresponding form field on the form.

If the partner later adds a new field to the form, the existing form entries will not be updated to include the new field. The partner can, however, create a new form entry for the client that includes the new field.

Likewise, if the partner removes or inactivates a form field, previously collected form field entries will still contain data for the removed field.

2.3.5. Reports and Data Dashboards



Reports in DMS are built on top of forms and pull data from the form entries associated with that form. A report transforms the scattered form field entries into a single table for display and dissemination. A report may include any or all of the fields currently associated with the form and may be filtered to include only a subset of the data (either statically or dynamically).

DRAFT

Data from the report may be aggregated (using familiar concepts such as grouping, summing, counting, averaging, etc.) and the aggregations may be used to populate charts such as pie charts, line charts, or bar charts.

Dashboards in the DMS are a way to organize and display aggregations and charts in a single view. A dashboard may include one or more dashboard components, each of which specifies an aggregation or chart and its size and location on the dashboard.

Relationships Between Entities

- A form may be associated with one or more reports.
- A report may be associated with one or more aggregations of the data.
- An aggregation may be associated with one or more charts.
- An aggregation or chart may be associated with one or more dashboard components.
- A dashboard component may be associated with one or more dashboards.

Example

A partner may create a report to track the progress of students in a program. The report may include fields for the student's name, the program name, the student's grade, and the student's attendance.

The partner may also create a dashboard to track the overall progress of the program. The dashboard may include a chart showing the average grade of students in the program and a chart showing the percentage of students who are attending regularly.

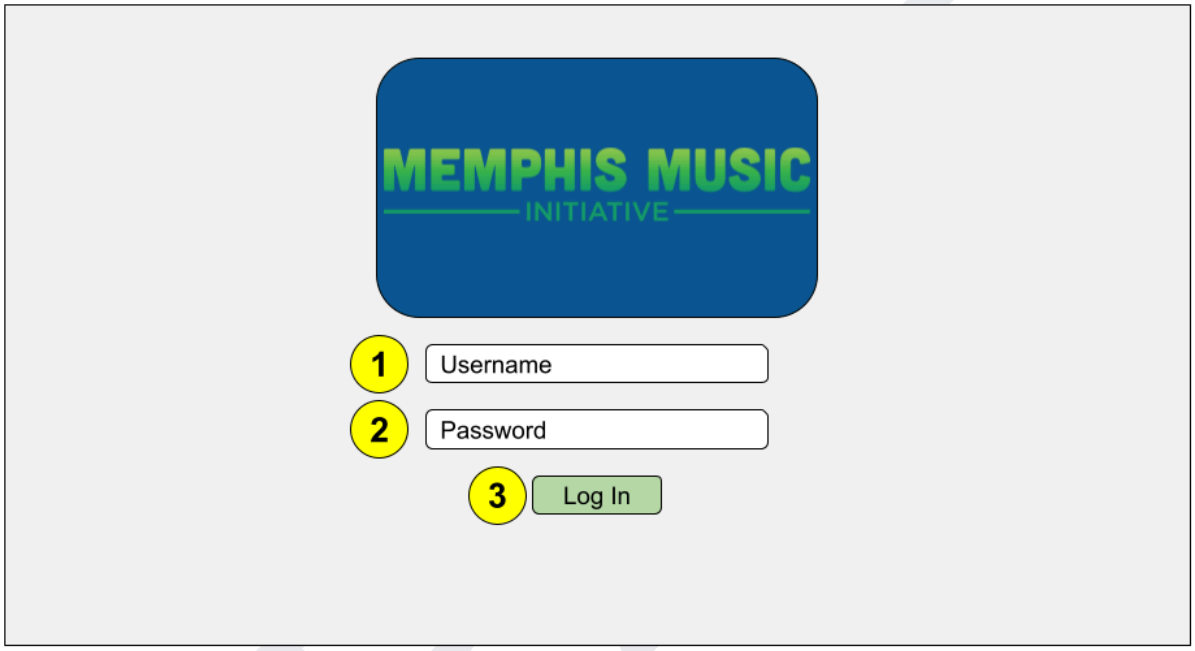
The dashboard component for the average grade chart may be associated with the report on student progress. The dashboard component for the attendance percentage chart may also be associated with the report on student progress.

DRAFT

2.4. User Interface

2.4.1. Login Screen

The login screen is the first and only screen accessible to any unauthenticated user. Here, users will be asked to enter their username and password in order to proceed to access the application.

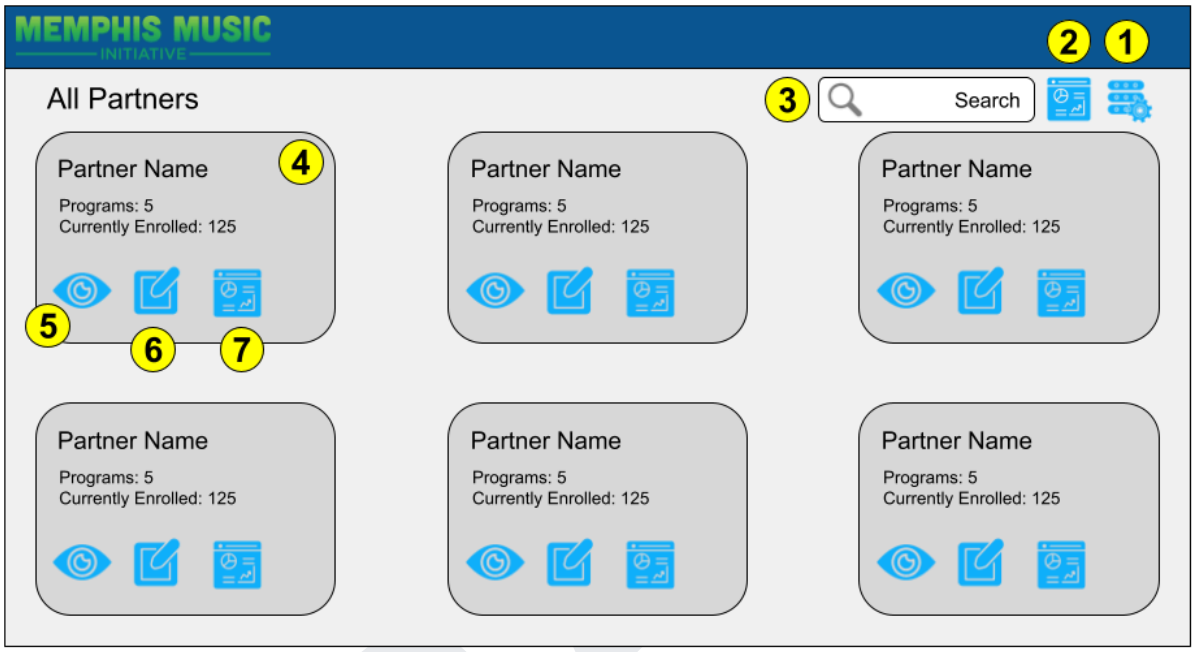


Interactive Elements

(1) Users enter their username into the username field.
(2) Users enter their password into the password field. Entered passwords are obfuscated during entry.
(3) This button will navigate the user to their home screen upon successful authentication. The specific home screen will depend on the user type.

2.4.2. MMI User Home Screen

The home screen for MMI Users is a list of active partners with controls for searching partners, adding/removing partners, adding users to partners, and viewing partner home screens.

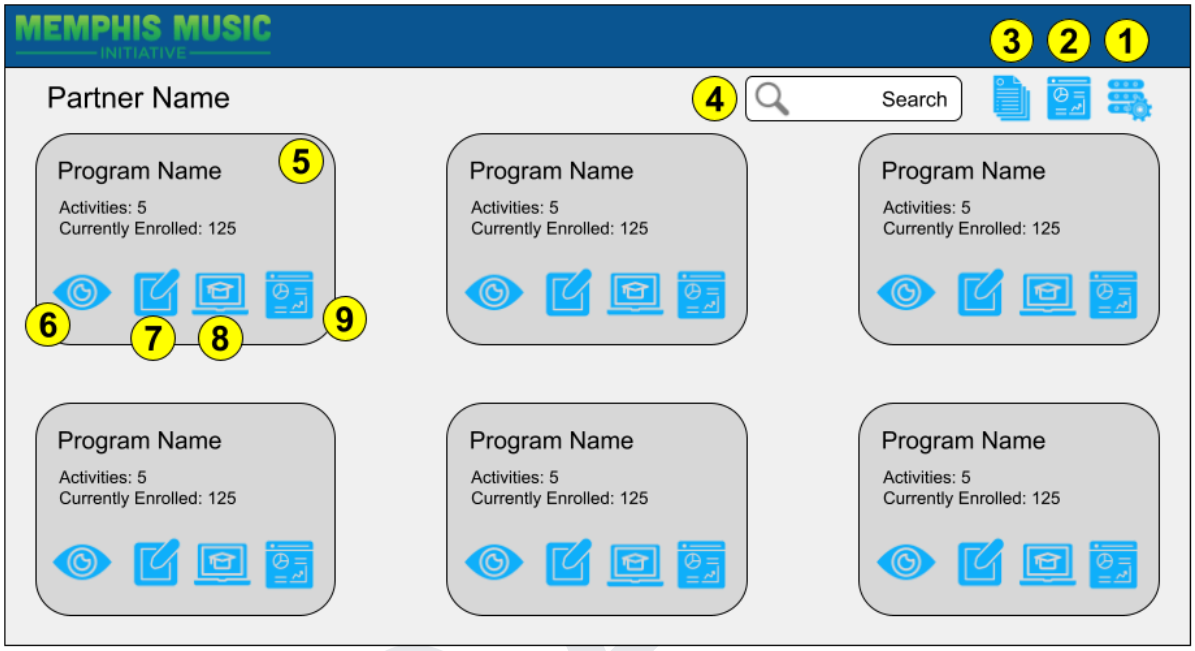


Interactive Elements

(1) This icon navigates to the Manage Partners screen.
(2) This icon navigates to the MMI Dashboard screen.
(3) This search bar can be used to filter the partners on screen by only displaying cards for partners whose name partially matches the search string.
(4) Each partner is listed in a card on screen which contains the partners name as well as some configurable metadata such as the number of active programs and the number of unique clients currently enrolled.
(5) This icon serves as a navigation control, navigating to the partner's home screen for viewing.
(6) This icon serves as a navigation control, navigating to the Edit Partner screen.
(7) This icon serves as a navigation control, navigating to the partner's Partner Dashboard screen.

2.4.3. Partner User Home Screen

The home screen for partner users is a list of programs with controls for searching programs, enrolling/removing clients, editing programs, and viewing program dashboards.

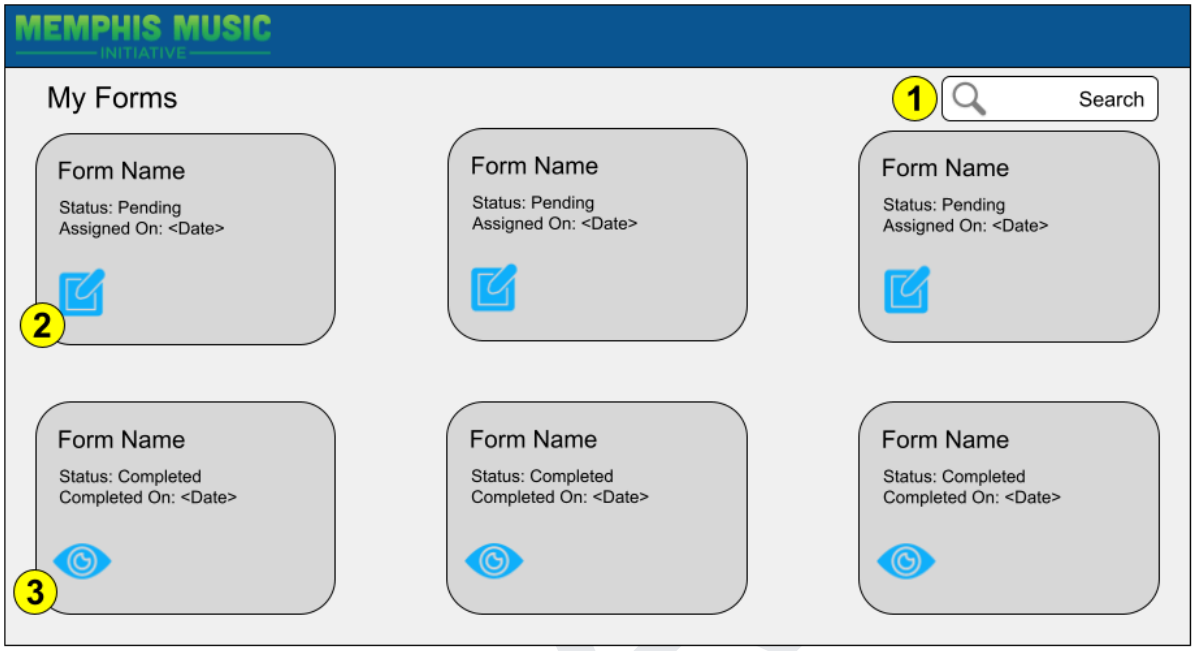


Interactive Elements

(1) This icon navigates to the Manage Programs screen.
(2) This icon navigates to the Partner Dashboard screen.
(3) This icon navigates to the Manage Forms screen.
(4) This search bar can be used to filter the programs on screen by only displaying cards for programs whose name partially matches the search string.
(5) Each program is listed in a card on screen which contains the program's name as well as some configurable metadata such as the number of active activities and the number of unique clients currently enrolled in that program.
(6) This icon serves as a navigation control, navigating to the Manage Program screen.
(7) This icon serves as a navigation control, navigating to the Edit Program screen.
(8) This icon serves as a navigation control, navigating to the Manage Activities screen.
(9) This icon serves as a navigation control, navigating to the program's Program Dashboard screen.

2.4.4. Parent and Client User Home Screens

The home screen for Client and Parent Users is a list of Forms that have been assigned either to the Client or to any Client who is a child of the Parent.



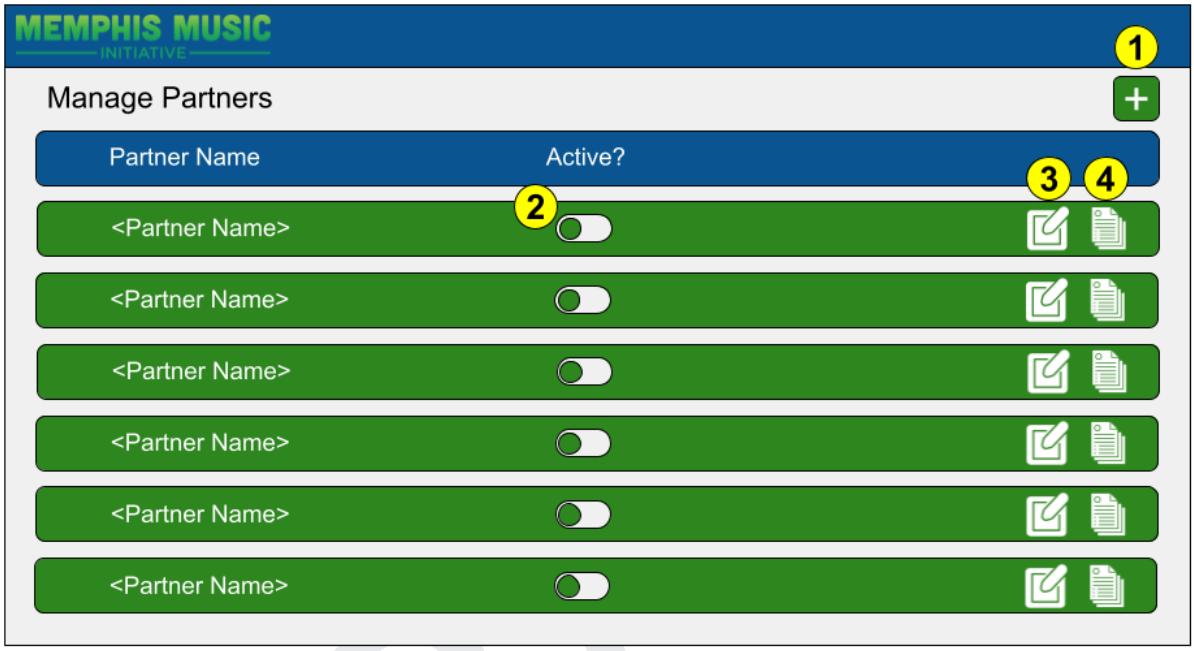
Interactive Elements

(1) This search bar can be used to filter the forms on screen by only displaying cards for forms whose name partially matches the search string.
(2) This icon navigates the user to the Complete Assigned Form screen.
(3) This icon navigates the user to the View Form Entry screen.

DRAFT

2.4.5. Manage Partners Screen

This screen displays all Partners in the system (active or inactive) in a tabular format with controls for setting the Partner's active status, editing/updating the Partner's information, and managing users associated with that Partner.

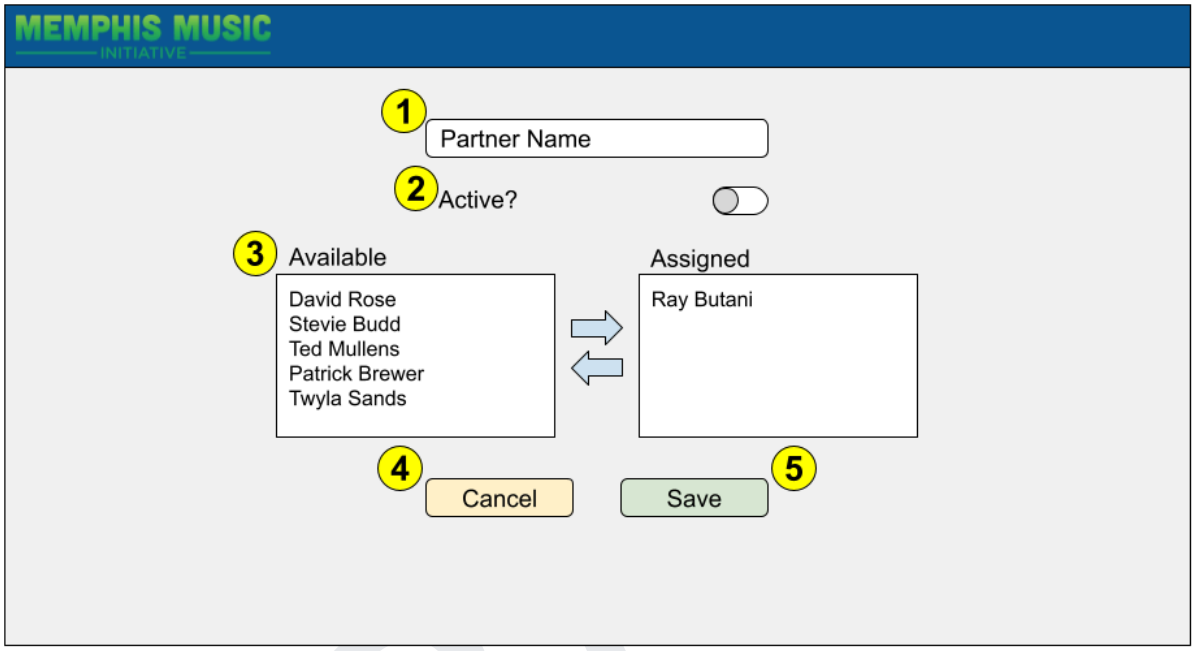


Interactive Elements

(1) This button navigates to the Add Partner screen.
(2) This toggle controls the Partner's active/inactive status.
(3) This icon navigates the user to the Edit Partner screen.
(4) This icon navigates the user to the Manage Forms screen.

2.4.6. Add/Edit Partner Screen

This screen serves as the form for adding a new Partner or updating an existing Partner. When adding a new Partner, the form fields will initially be blank. When updating an existing partner, the form fields will be filled with that Partner's information.

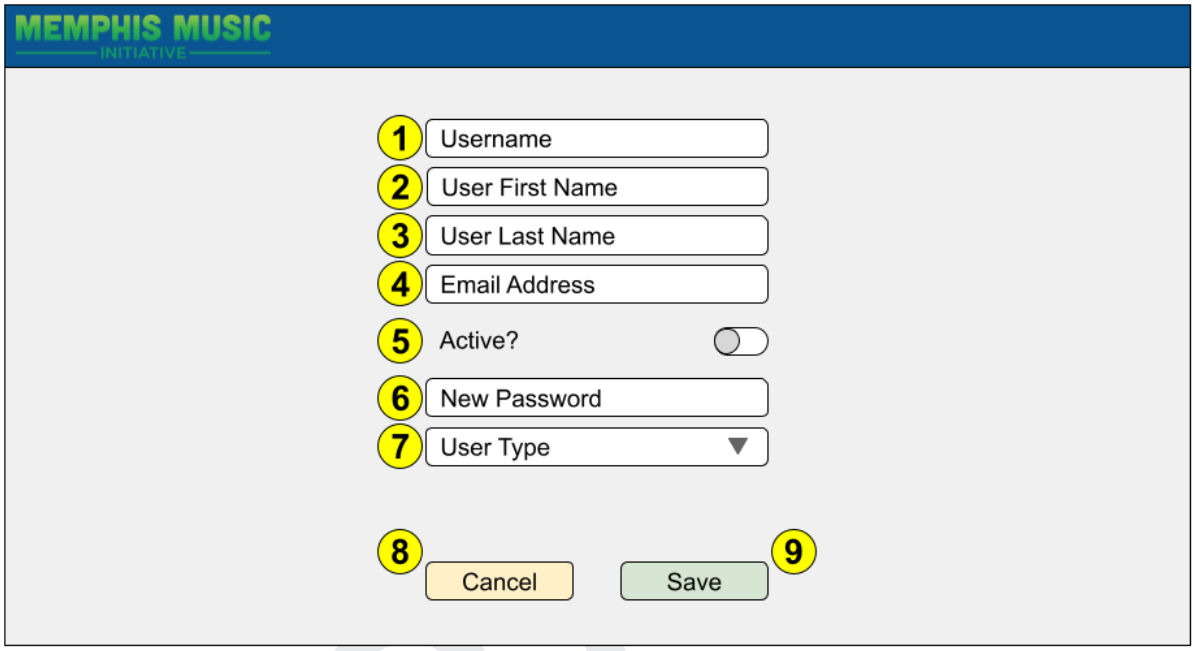


Interactive Elements

(1) This field is used to add/update the Partner's name.
(2) This toggle is used to set the Partner's active/inactive status.
(3) This control is used to add/remove users to the Partner. The right arrow button moves a user from "Available" to "Assigned" and the left arrow moves a user from "Assigned" to "Available". A user may only be assigned to one Partner.
(4) This button is used to cancel the operation and return to the previous screen.
(5) This button is used to save the current Partner's information.

2.4.7. Add/Edit User Screen

This screen serves as the form for adding a new User or updating an existing User. When adding a new User, the form fields will initially be blank. When updating an existing User, the form fields will be filled with that User's information.

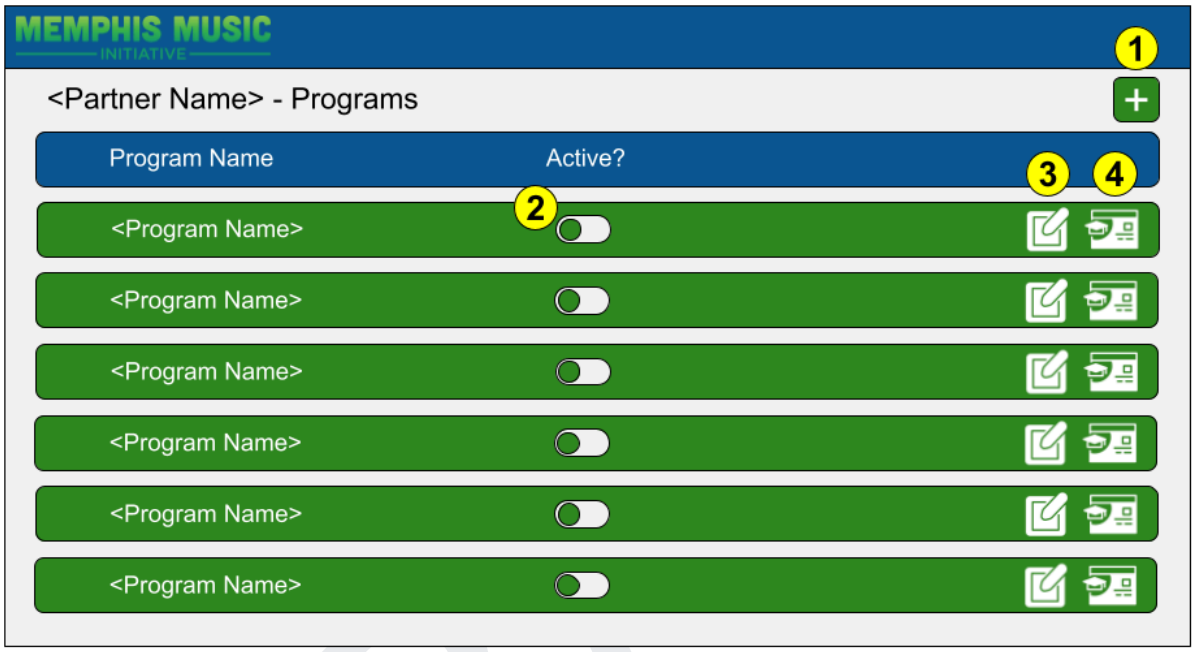


Interactive Elements

(1) This field is used to add/update the User's username.
(2) This field is used to add/update the User's first name.
(3) This field is used to add/update the User's last name.
(4) This field is used to add/update the User's email address.
(5) This toggle is used to set the Partner's active/inactive status.
(6) This field is used to set a temporary password for the User.
(7) This field is used to set the type of User.
(8) This button is used to cancel the operation and return to the previous screen.
(9) This button is used to save the current User's information.

2.4.8. Manage Programs Screen

This screen displays all Programs associated with the current partner in the system (active or inactive) in a tabular format with controls for setting the Program's active status, editing/updating the Program's information, managing Activities associated with that Program, and managing Clients associated with that Program.

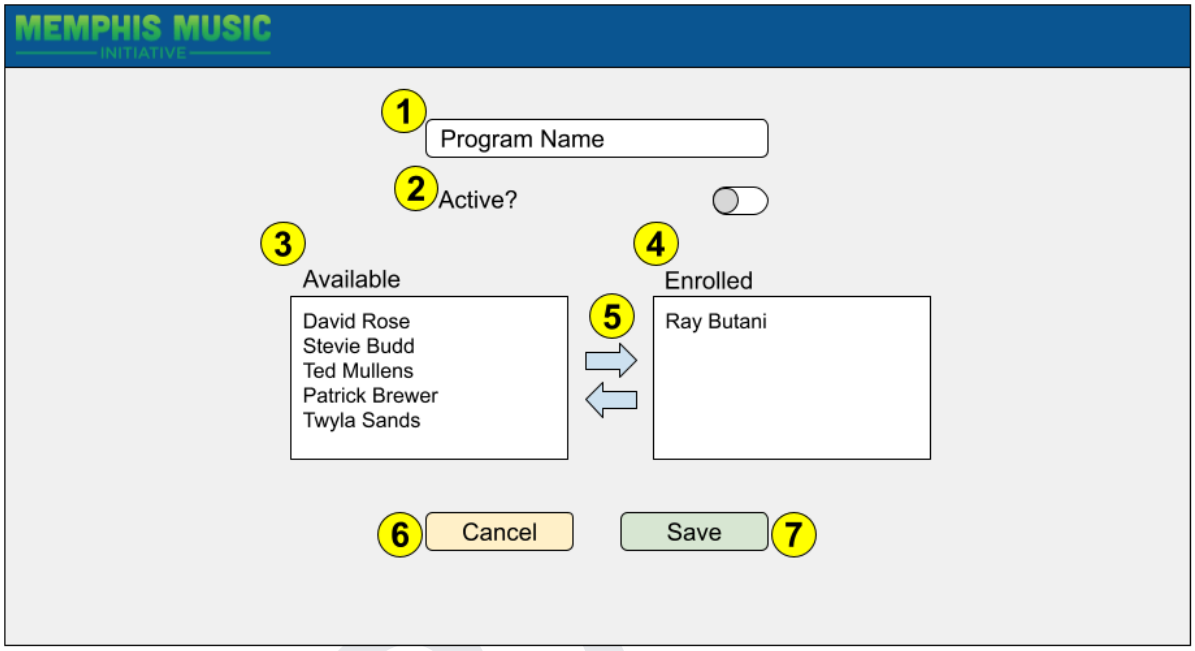


Interactive Elements

(1) This button navigates to the Add Program screen.
(2) This toggle controls the Program's active/inactive status.
(3) This icon navigates the user to the Edit Program screen.
(4) This icon navigates the user to the Manage Activities screen.

2.4.9. Add/Edit Program Screen

This screen serves as the form for adding a new Program or updating an existing Program. When adding a new Program, the form fields will initially be blank. When updating an existing Program, the form fields will be filled with that Program's information.

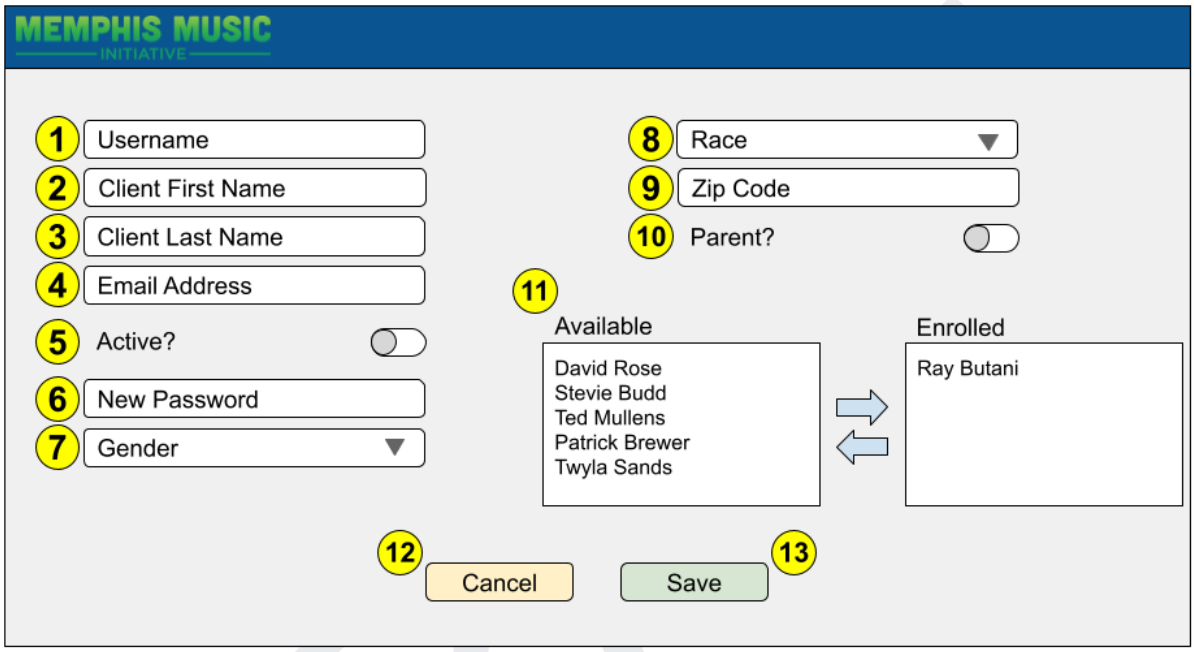


Interactive Elements

(1) This field is used to add/update the Program's name.
(2) This toggle is used to set the Program's active/inactive status.
(3) This field lists the Client users that are available to be enrolled in the Program.
(4) This field lists the Client users that are already enrolled in the Program.
(5) These arrow buttons allow adding Clients to and removing Clients from the Program.
(6) This button is used to cancel the operation and return to the previous screen.
(7) This button is used to save the current Program's information.

2.4.10. Add/Edit Client Screen

This screen serves as the form for adding a new Client/Parent or updating an existing Client/Parent. When adding a new Client/Parent, the form fields will initially be blank. When updating an existing Client/Parent, the form fields will be filled with that Client/Parent's information.

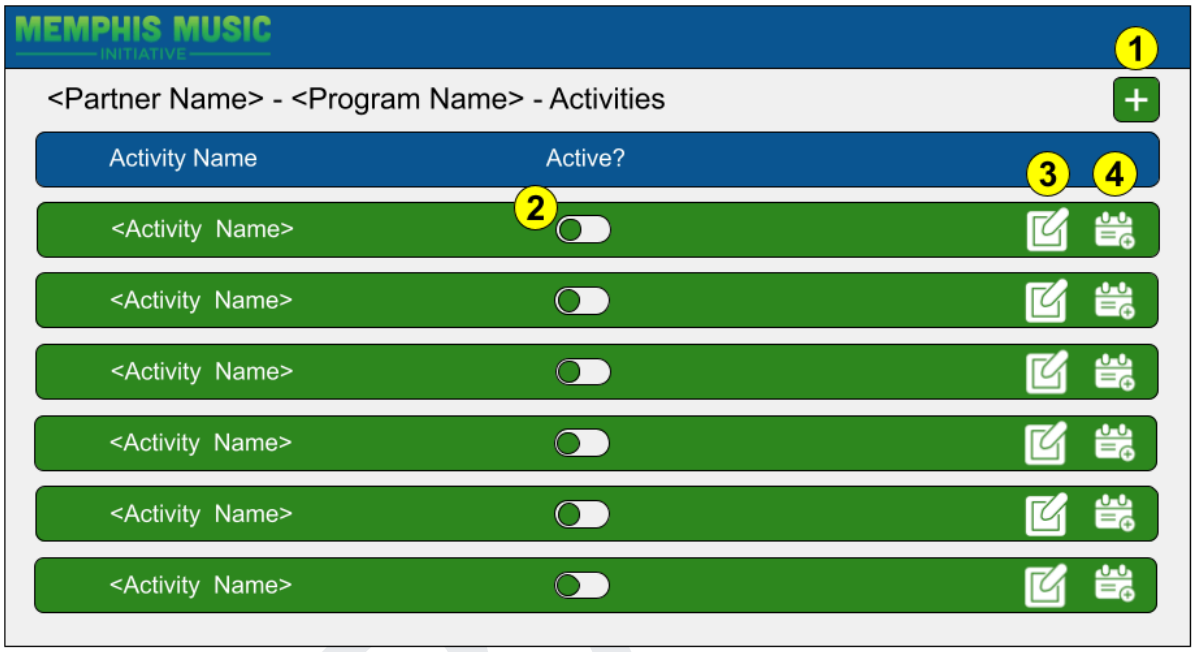


Interactive Elements

(1) This field is used to add/update the Client's username.
(2) This field is used to add/update the Client's first name.
(3) This field is used to add/update the Client's last name.
(4) This field is used to add/update the Client's email address.
(5) This toggle is used to set the Client's active/inactive status.
(6) This field is used to set a temporary password for the Client.
(7) This field is used to set the Client's gender.
(8) This field is used to set the Client's race.
(9) This field is used to set the Client's zip code.
(10) This field is used to toggle whether or not this client is a Parent.
(11) This optional field is used to indicate which other Client's are under the care of this Parent.
(12) This button is used to cancel the operation and return to the previous screen.
(13) This button is used to save the current Client's information.

2.4.11. Manage Activities Screen

This screen displays all Activities associated with the current program in the system (active or inactive) in a tabular format with controls for setting the Activity's active status, editing/updating the Activity's information, managing Sessions associated with that Activity, and managing Clients associated with that Activity.

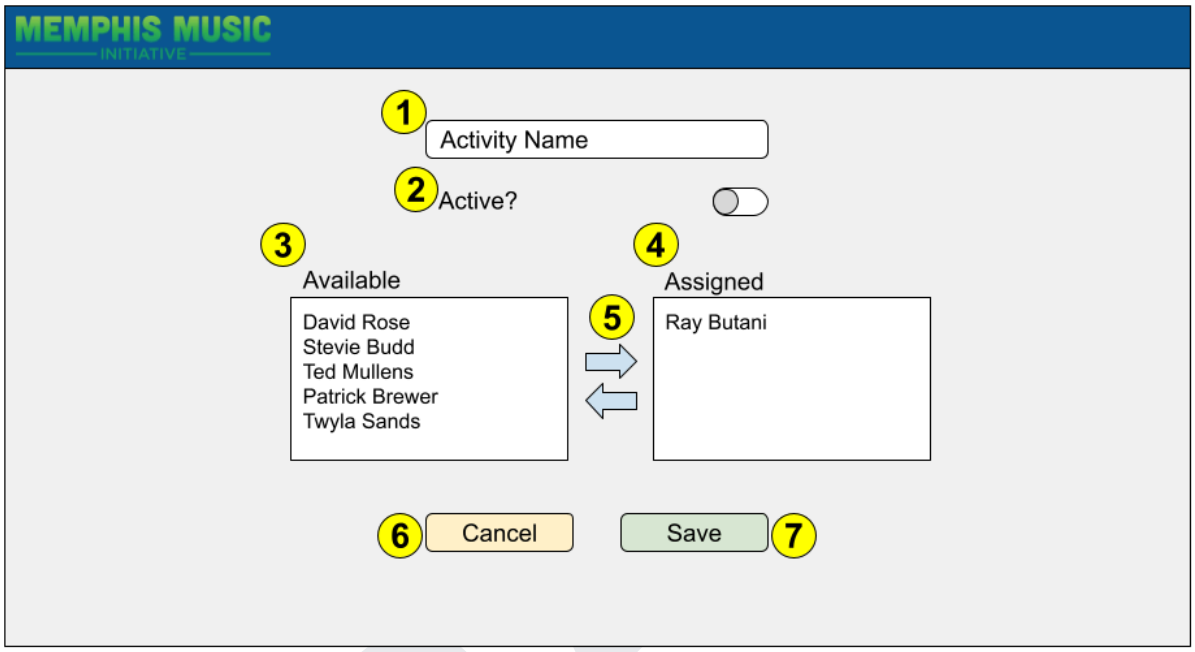


Interactive Elements

(1) This button navigates to the Add Activity screen.
(2) This toggle controls the Activity's active/inactive status.
(3) This icon navigates the user to the Edit Activity screen.
(4) This icon navigates the user to the Manage Sessions screen.

2.4.12. Add/Edit Activity Screen

This screen serves as the form for adding a new Activity or updating an existing Activity. When adding a new Activity, the form fields will initially be blank. When updating an existing Activity, the form fields will be filled with that Activity's information.

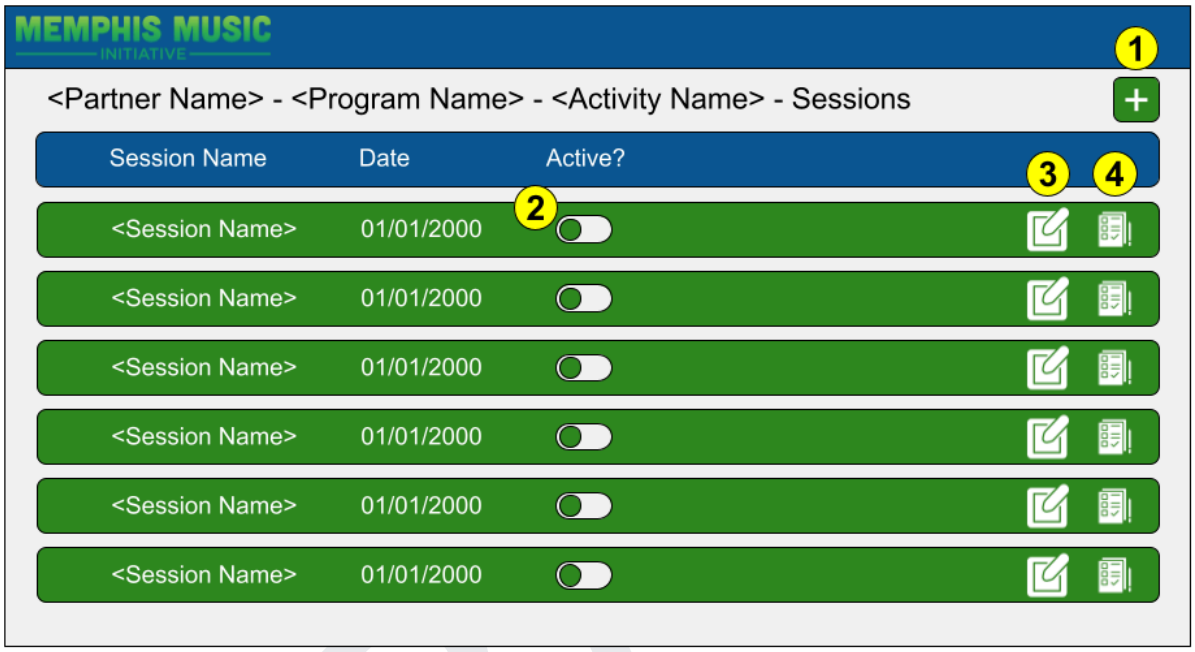


Interactive Elements

(1) This field is used to add/update the Activity's name.
(2) This toggle is used to set the Activity's active/inactive status.
(3) This list includes all Clients who are enrolled in the related Program.
(4) This list includes all Clients assigned to this Activity. Will default to all clients enrolled in the Program.
(5) These arrow buttons can be used to assign/unassign Clients to this Activity.
(6) This button is used to cancel the operation and return to the previous screen.
(7) This button is used to save the current Activity's information.

2.4.13. Manage Sessions Screen

This screen displays all Sessions associated with the current activity in the system (active or inactive) in a tabular format with controls for setting the Session's active status, editing/updating the Session's information, and managing Clients associated with that Session.

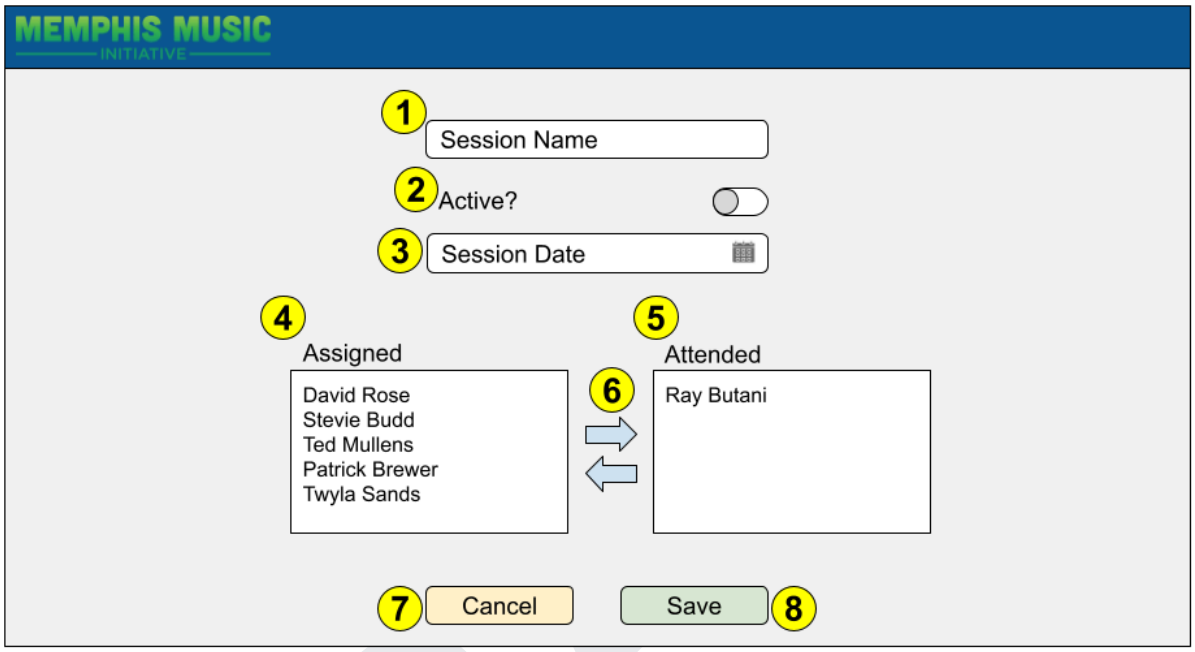


Interactive Elements

(1) This button navigates to the Add Session screen.
(2) This toggle controls the Session's active/inactive status.
(3) This icon navigates the user to the Edit Session screen.
(4) This icon navigates the user to the Assign Form screen.

2.4.14. Add/Edit Session Screen

This screen serves as the form for adding a new Session or updating an existing Session. When adding a new Session, the form fields will initially be blank. When updating an existing Session, the form fields will be filled with that Session's information.

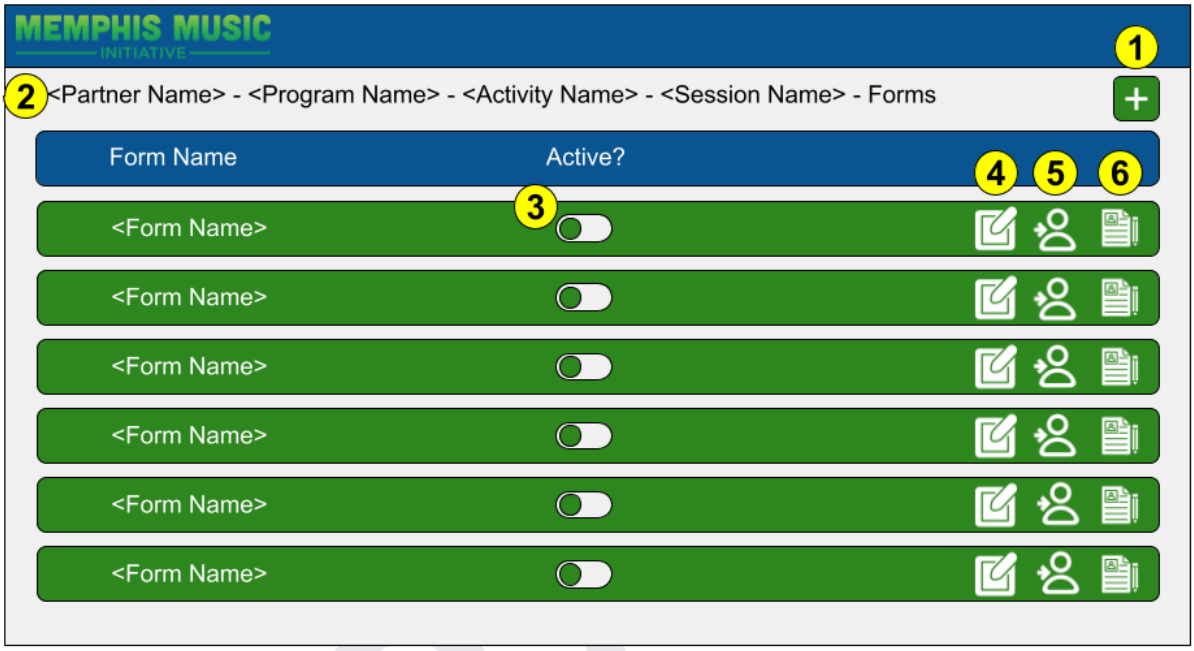


Interactive Elements

(1) This field is used to add/update the Session's name.
(2) This toggle is used to set the Session's active/inactive status.
(3) This field is used to enter the date of the Session.
(4) This field lists all the Clients assigned to the Activity associated with this Session.
(5) This field lists all the Clients who attended this Session.
(6) These arrow buttons can be used to move clients between the Assigned and Attended lists.
(7) This button is used to cancel the operation and return to the previous screen.
(8) This button is used to save the current Session's information.

2.4.15. Manage Forms Screen

This screen displays all forms associated with a Partner with controls for creating a new Form, editing an existing Form, associating Forms with Programs, Activities, and Sessions, and viewing Form Entries.

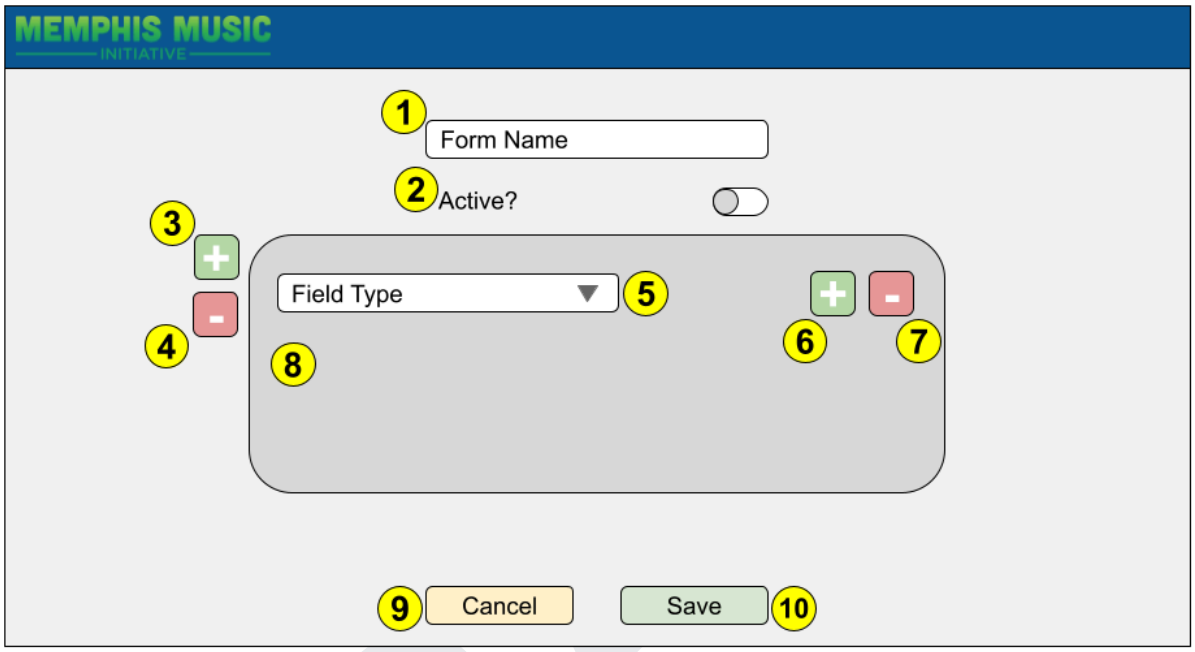


Interactive Elements

(1) This button navigates to the Add Form screen.
(2) This display will indicate the associations for this Form. The exact information displayed will change based on the entity whose Forms are being listed on this screen. For example, when viewing Forms for a Program, only the Program and associated Partner names will be displayed.
(3) This toggle controls the Form's active/inactive status.
(4) This icon navigates the user to the Edit Form screen.
(5) This icon navigates the user to the Form Assignment screen.
(6) This icon navigates the user to the Complete Assigned Form screen.

2.4.16. Add/Edit Form Screen

This screen serves as the form for adding a new Form or updating an existing Form. When adding a new Form, the form fields will initially be blank. When updating an existing Form, the form fields will be filled with that Form's information.

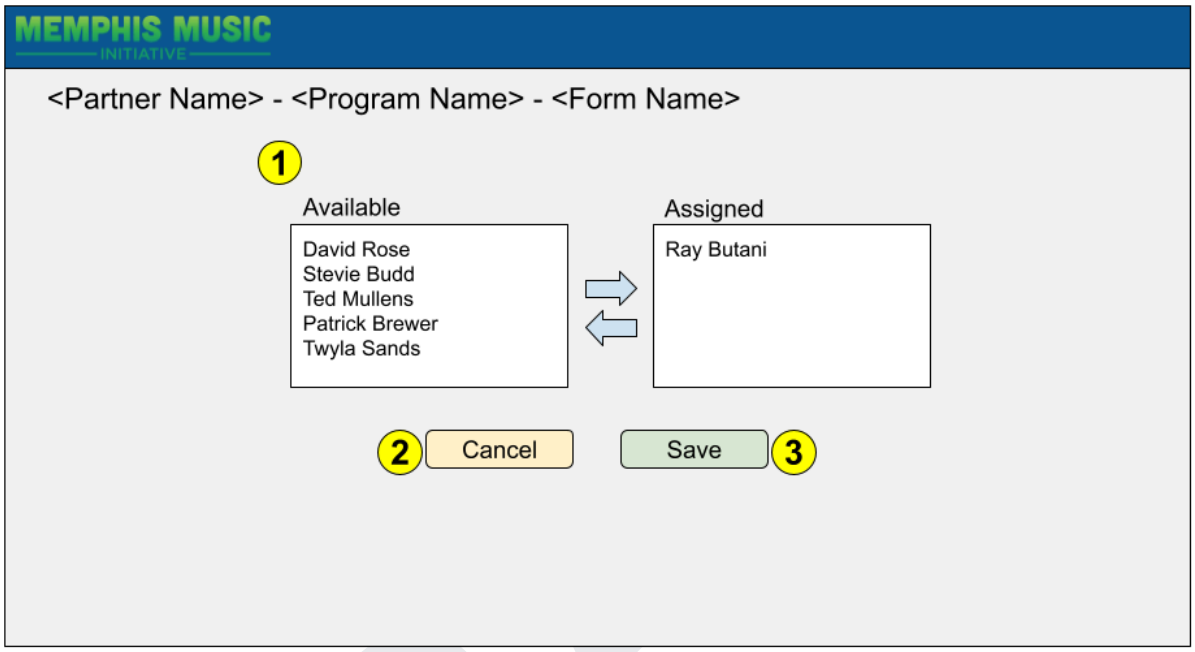


Interactive Elements

(1) This field is used to add/update the Form Section's name
(2) This toggle sets the Form's active/inactive status.
(3) This button adds a new Form Section to the Form.
(4) This button is used to remove a Form Section from the Form.
(5) This dropdown is used to select which type of Form Field to add.
(6) This button is used to add a Form Field to the Form Section.
(7) This button is used to remove a Form Field from a Form.
(8) Additional fields allowing adding/editing data associated with each Form Field type will appear in this space.
(9) This button is used to cancel the operation and return to the previous screen.
(10) This button is used to save the current Form's information.

2.4.17. Form Assignment Screen

This screen displays a list of all Clients associated with the same Partner, Program, Activity, or Session as the Form being assigned. The Form can be assigned to some, all, or none of those Clients.

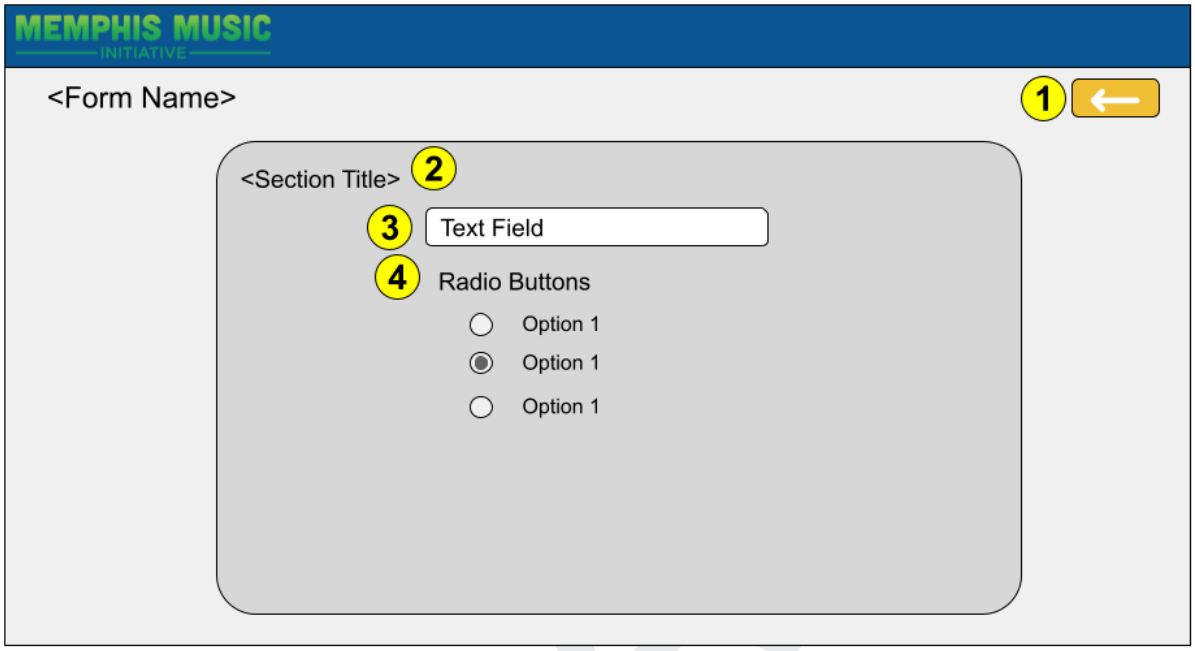


Interactive Elements

(1) This control is used to add/remove clients to the assignment list.
(2) This button assigns the selected Form to the selected Clients.
(3) This button cancels the operation and returns to the previous screen.

2.4.18. View Form Entry

This screen displays a completed Form Entry, but does not allow editing of that Form Entry.



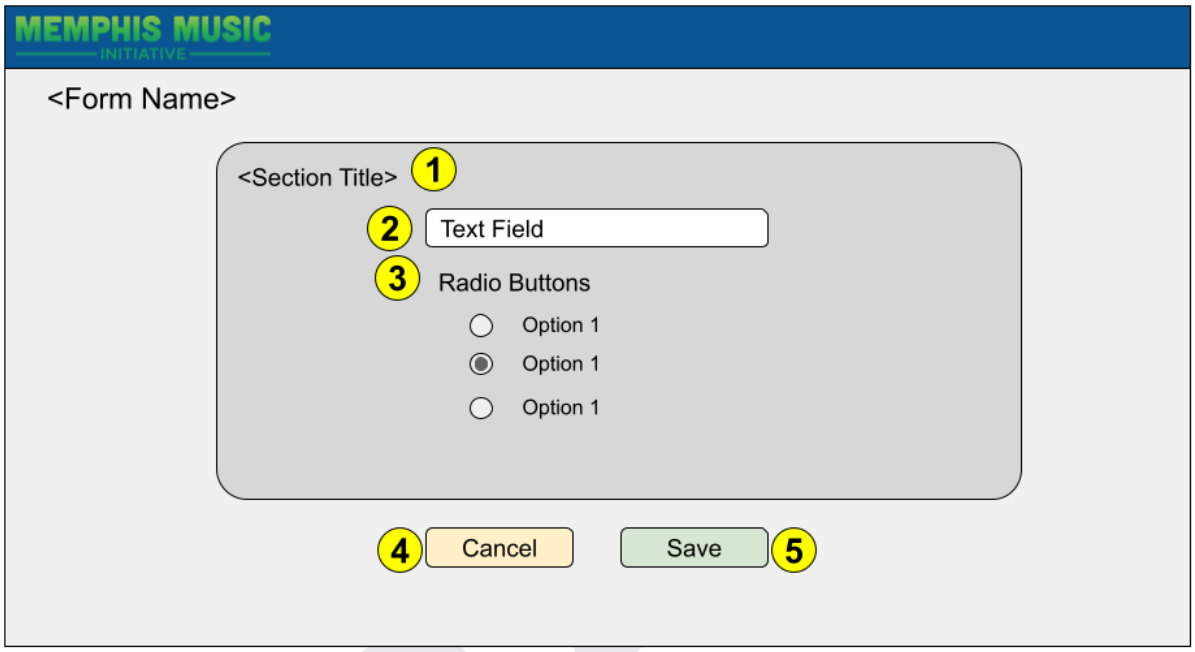
Interactive Elements

(1) This button navigates the user to the previous screen.
(2) Each Form Section is displayed.
(3) This is an example filled text field.
(4) This is an example filled choice field (radio buttons).

DRAFT

2.4.19. Complete Assigned Form Screen

This screen displays a fillable Form to the Client who has been assigned that Form for completion. The wireframe below indicates an example Form, but the format will change based on which Form Sections and Form Fields are in the Form.

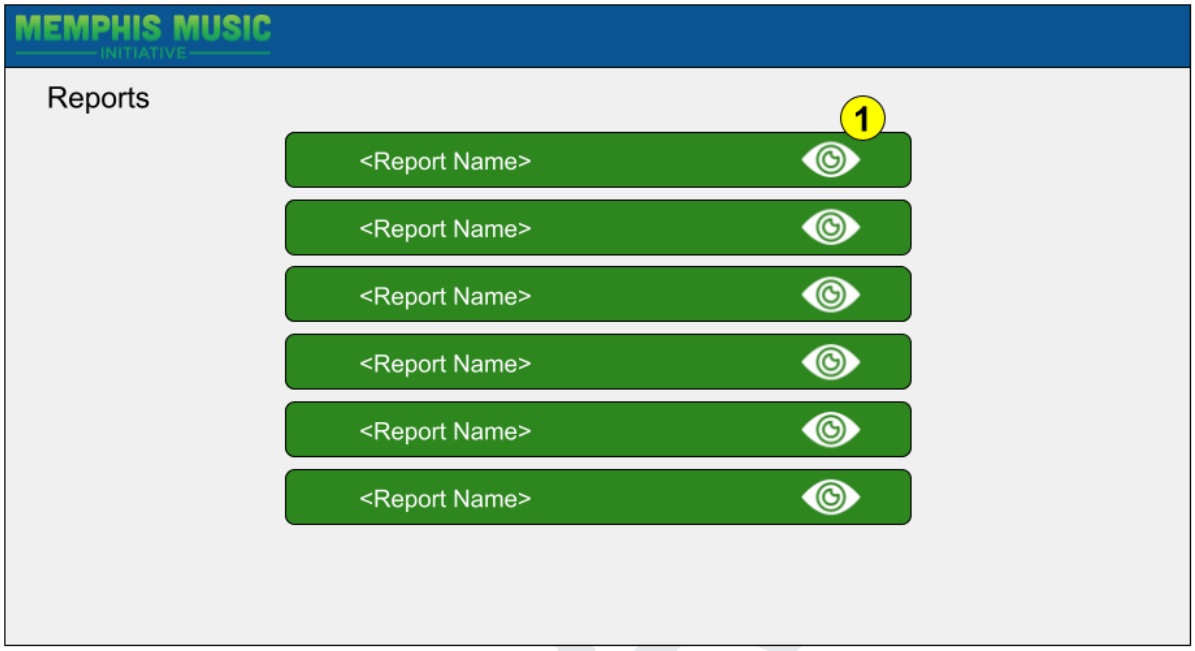


Interactive Elements

(1) Each Form Section is listed with a label.
(2) This is an example, fillable text field.
(3) This is an example, fillable choice field (radio buttons).
(4) This button cancels the operation and returns to the previous screen.
(5) This button saves the completed form.
(6) This drop-down is only displayed when a Partner User is completing a Form on behalf of a Client. Used to select the client to associate the Form with.

2.4.20. Reports Menu

This screen displays a list of reports that are accessible by the user, with links to access each report.



Interactive Elements

(1) This icon navigates the user to the View Report screen for the Report.

DRAFT

2.4.21. View Report Screen

This screen displays a chosen Report.

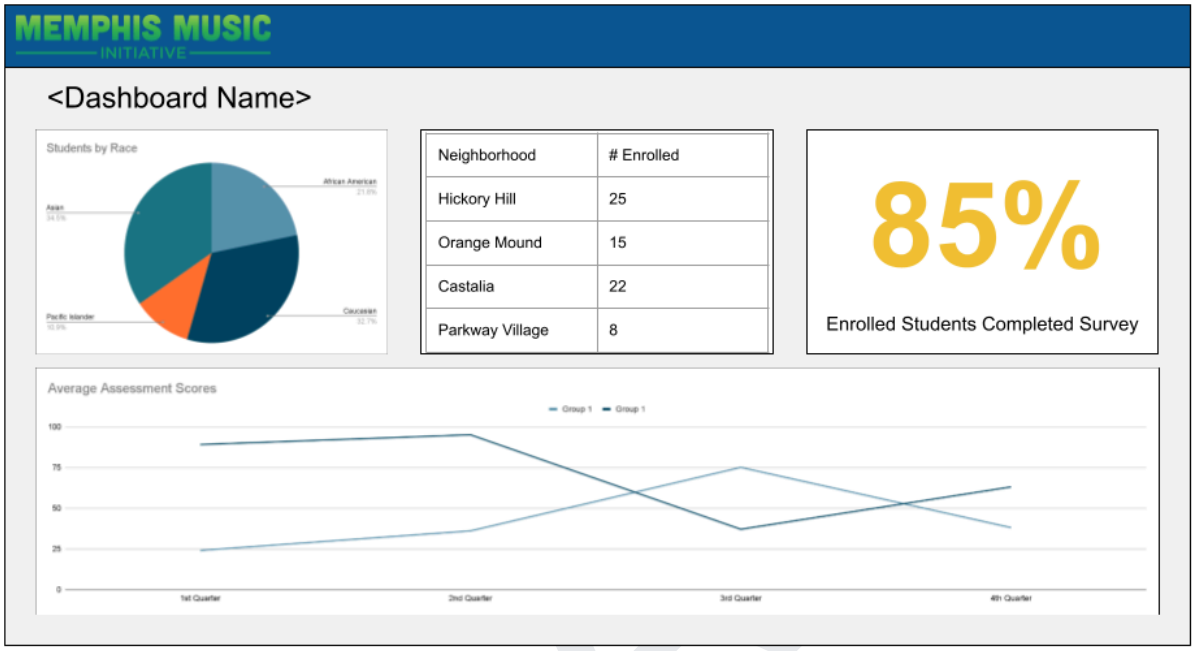
Interactive Elements

(1) This icon downloads the displayed report.
(2) This icon prints the displayed report.
(3) This icon triggers a dialog to send the displayed report via e-mail.

DRAFT

2.4.22. MMI/Partner/Program Dashboard Screen

This screen displays a Dashboard, consisting of a number of configurable elements. The wireframe below demonstrates a possible configuration.

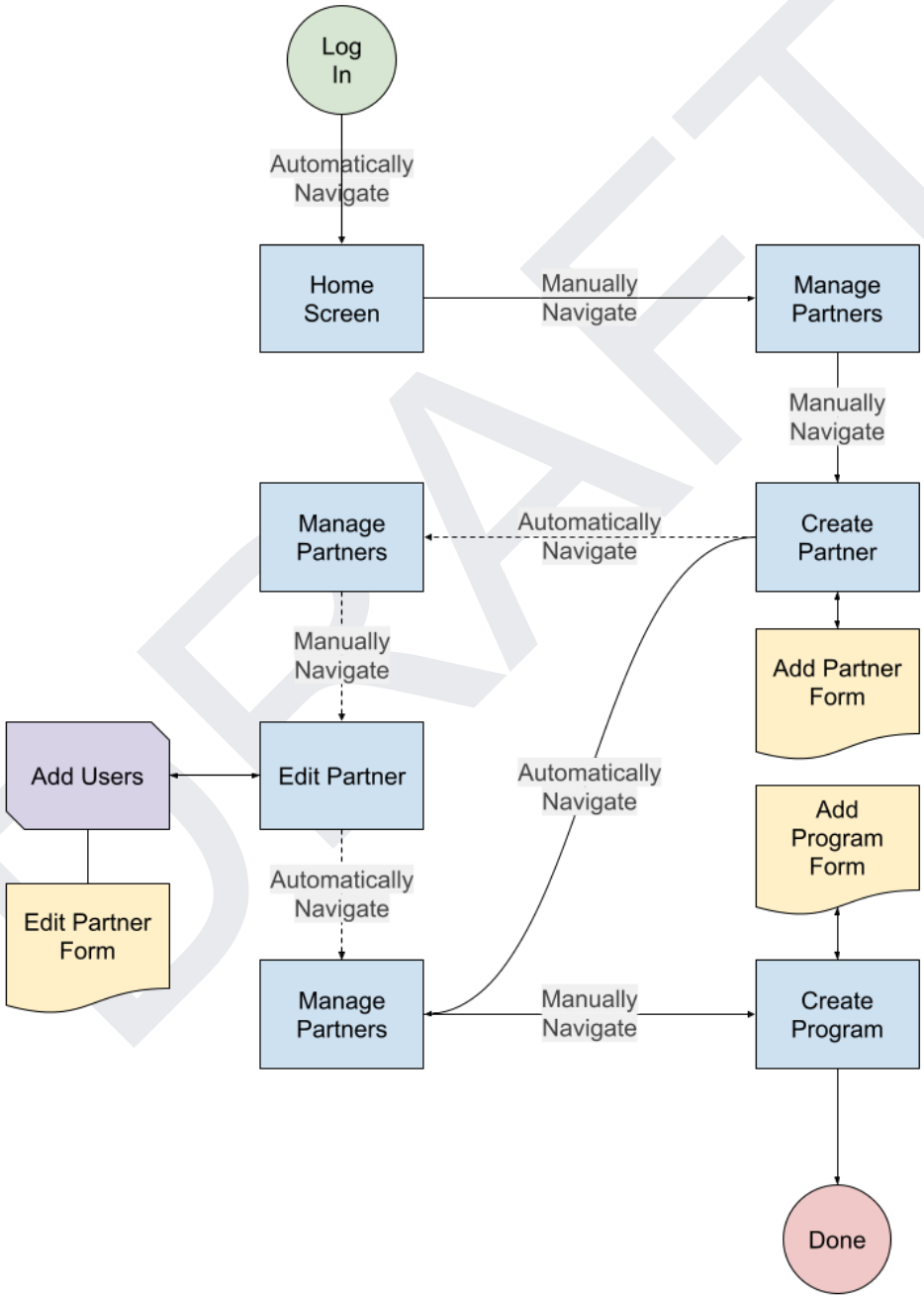


DRAFT

2.5. Sample Workflows

2.5.1. Setting up a New Partner and Program

An MMI User may occasionally need to add a new MMI Partner (or unit for collecting data) with one or more Programs. This workflow explores the process of creating that new Partner, adding users to that Partner, and creating a new Program.



DRAFT

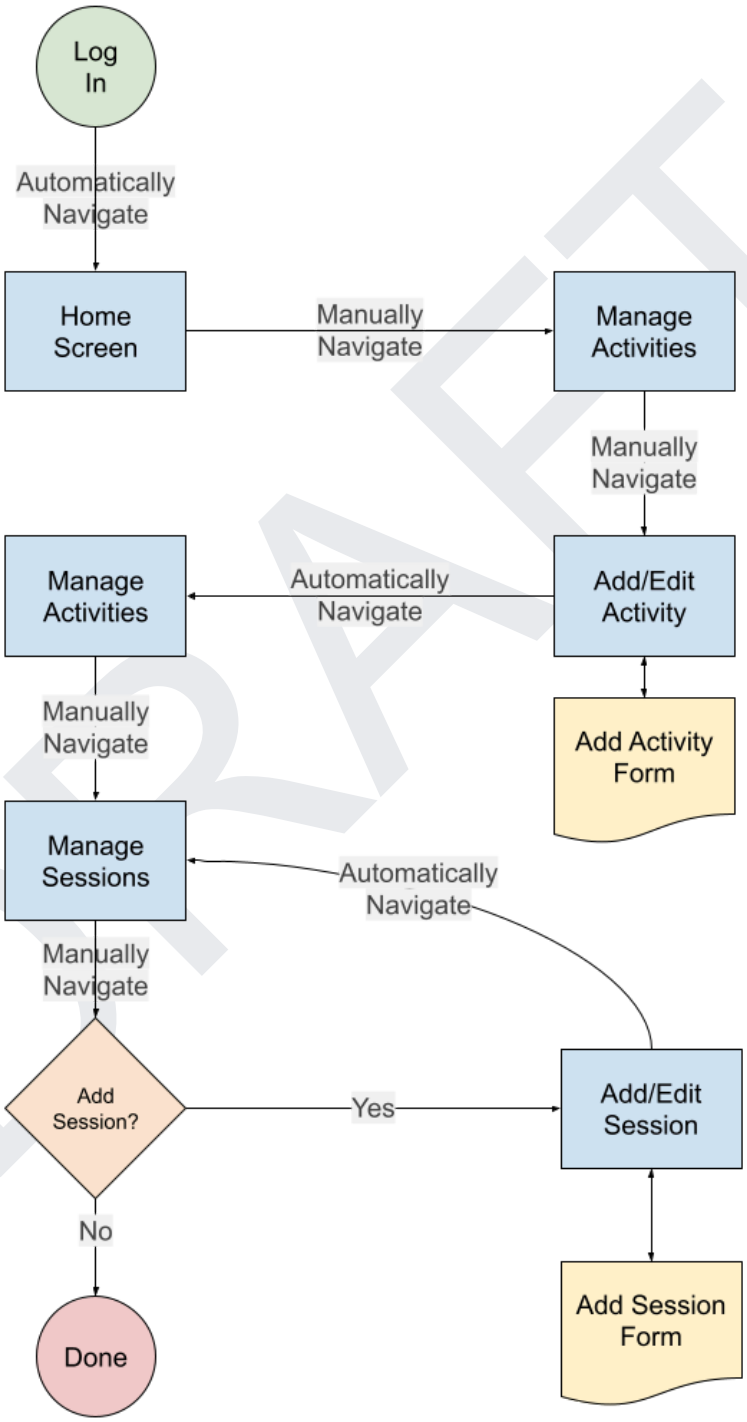
Workflow

(1) An MMI User logs into the DMS and is automatically navigated to their home screen.
(2) From the home screen, the user clicks the icon to navigate to the Manage Partners screen.
(3) The user creates a new Partner by filling out the form presented on that screen and clicking the “Save” button, automatically navigating back to the Manage Partners screen.
(4) (Optional) If the user forgot to add Users to the Partner, or the Users weren’t created yet.
    (a) The user identifies the newly created Partner on the Manage Partners screen and clicks the icon to navigate to the Edit Partner screen.
    (b) The user associates users with the Partner and clicks the “Save” button, automatically navigating back to the Manage Partners screen.
(5) The user clicks the icon to add a Program to the Partner on the Manage Partners screen.
(6) The user creates the new Program, which is automatically associated with the Partner.

DRAFT

2.5.2. Adding Activities and Sessions to a Program

A Partner User will frequently wish to add Activities or Session to a Program in order to track participation by Clients. This workflow demonstrates the process of adding an Activity and one or more Sessions for the new Activity.



DRAFT

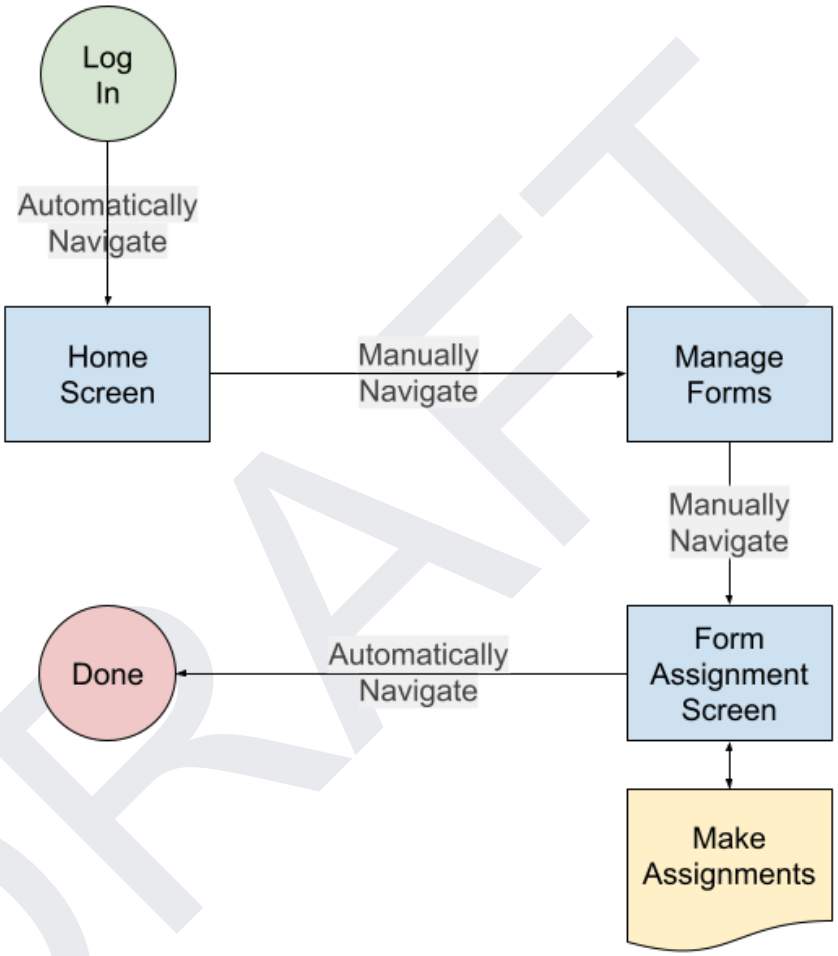
Workflow

(1) A Partner User logs into the DMS and is automatically navigated to their home screen.
(2) That user selects the Manage Activities icon for the program they wish to add activities to, navigating that user to the Manage Activities screen for the chosen program.
(3) The user clicks the Add Activity button, navigating the user to the Add/Edit Activity screen. The user fills out the Activity information and clicks Save, automatically navigating the user back to the Manage Activities screen.
(4) The user selects the Manage Sessions icon for the newly created Activity in the Manage Activities screen, navigating that user to the Manage Sessions screen for that Activity.
(5) The user clicks the Add Session button, navigating the user to the Add/Edit Session screen. The user fills out the Session information and clicks Save, automatically navigating the user back to the Manage Sessions screen.
(6) If the user wishes to add more Sessions, repeat step 5.

DRAFT

2.5.3. Conducting a Retrospective Survey

Since Forms are the main method for collecting Client data, the DMS makes assigning Forms for completion extremely simple. The workflow shown here demonstrates assigning a Form associated with a Program, however, the general approach holds the same for Forms assigned to other organizational elements (Activities and Sessions).



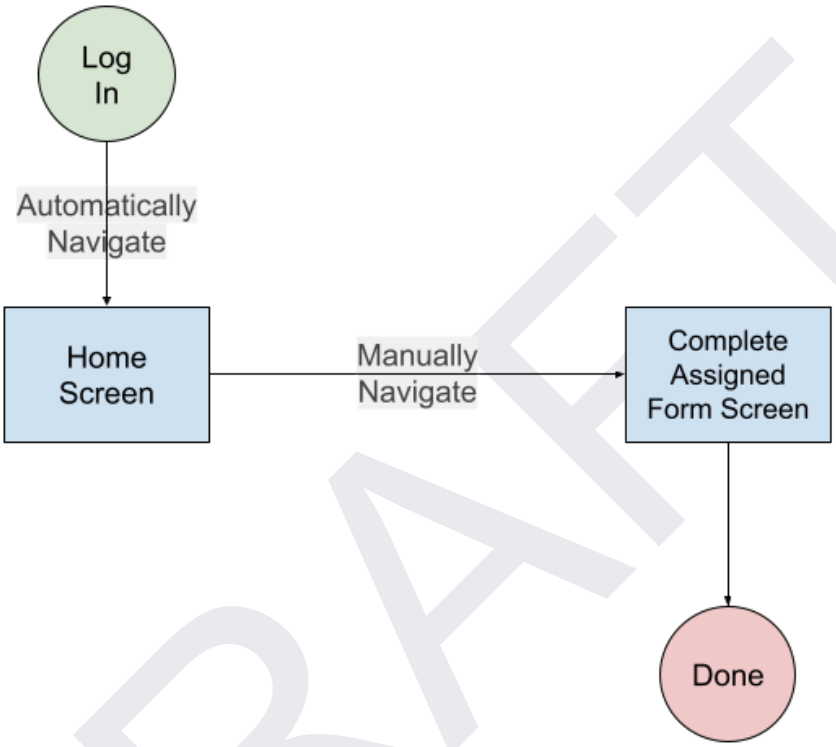
Workflow

(1) A Partner User logs into the DMS and is automatically navigated to their home screen.
(2) The user selects the Manage Forms icon for the Program they wish to collect the survey for.
(3) The user assigns the Form to some or all of the Clients associated with that Program.

DRAFT

2.5.4. Completing a Form (Client/Parent)

In addition to ease of assigning Forms, completing Forms is also a straightforward process for Clients and Parents. In brief, all assigned Forms (completed or not) appear on the Client/Parent home screen, from which the Parent/Client can select a Form to complete.



Workflow

(1) A Client/Partner User logs into the DMS and is automatically navigated to their home screen.
(2) The user selects the Complete Form icon for the Form the user wishes to complete, navigating them to the Complete Assigned Form screen for that Form.
(3) The user completes and saves the form, which returns them to their home screen.

DRAFT

2.6. Security Measures

The security features of the DMS include:

- Use of Traefik as a reverse proxy: Traefik can be used to terminate TLS connections, perform authentication and authorization, and route traffic to the appropriate services. This helps to protect the underlying services from direct exposure to the internet.
- Use of HTTPS: All traffic to the application is served over HTTPS, which encrypts the data in transit.
- Use of Let's Encrypt for SSL certificates: Let's Encrypt provides free SSL certificates, which can be used to secure the application without having to manually manage and renew certificates.
- Use of basic authentication for the Traefik dashboard: This helps to protect the dashboard from unauthorized access.
- Use of health checks: Health checks are used to ensure that the services are running and healthy. If a service fails a health check, it will be restarted.
- Use of a dedicated network for internal communication: The services communicate with each other over a dedicated network, which helps to isolate them from other traffic on the host system.

Because the application is composed of microservices orchestrated via Docker Swarm mode, the following best practices are in use:

- Use of separate services for different components of the application: This helps to isolate the components from each other and makes it easier to manage their security.
- Use of volumes for persistent data: This helps to ensure that the persistent data is not lost if a container is restarted or deleted.
- Use of environment variables for sensitive data: This helps to prevent sensitive data from being stored in the container images.

In addition, the backup feature of the Linode VPS hosting platform will be utilized to ensure that the application is able to recover from even catastrophic failure in the event that the physical devices hosting the application go offline.

DRAFT

2.7. Scalability and Performance

The application is scalable because it is composed of multiple services. Each service can be scaled independently by increasing or decreasing the number of replicas. This allows the application to handle increased traffic without impacting performance.

The application also uses a dedicated network for internal communication. This helps to isolate the services from each other and makes it easier to scale the application.

The application is performant because it uses a number of performance optimizations, such as:

- Use of a reverse proxy: Traefik can be used to cache static content and offload TLS termination from the application servers. This can improve the performance of the application, especially for high-traffic websites.
- Use of a dedicated network for internal communication: Isolating the services from each other can improve performance by reducing network congestion.
- Use of health checks: Health checks can be used to identify and restart unhealthy services quickly. This helps to maintain the performance of the application even if some services fail.

The actual scalability and performance of the application will depend on a number of factors, such as the hardware configuration of the host system, the number of replicas, and the type of traffic. However, the use of orchestrated microservices hosted on a Linode VPS allows system administrators to scale up both the capabilities of individual servers and the number of service replicas to meet demand. If desired, the system may also be hosted across multiple VPS's in the same data center to further increase resiliency for added hosting costs.

Some example of architecture scaling include:

1. In cases where the number of requests per second to the application stresses the ability of the Django application service to respond quickly, the number of instances of the Django application can be increased on both the original VPS host as well as across multiple VPS hosts. The Traefik load balancer will support seamless replication, such that which specific replica is accessed by an end user can remain invisible and may even change during a single session without the end user becoming aware.
2. As architected, if the number of requests to the database per second begins to exceed the capacity of the Postgres database, the most straightforward scaling approach would be to increase the CPUs dedicated to the host VPS for the database microservice. Accordingly, if

DRAFT

the data volume begins to occupy too significant a fraction of available storage, the storage volume of the VPS server can be increased. In base configuration () the database is rated for approximately 10,000 requests per second. Additional hardware resources can increase this capacity by at least an order of magnitude (i.e., approximately 100,000 requests per second).

DRAFT